國立臺灣大學電機資訊學院資訊工程研究所

碩士論文

Department of Computer Science an Information Engineering

College of Electronical Engineering and Computer Science

National Taiwan University

Master's Thesis

應用於邊緣運算環境之低延遲 Wasm 執行時期分層編譯架構

A Tiered Compilation Architecture for Low-Latency Wasm Runtimes in Edge Computing Environments

李祥宇

Hsiang-Yu Lee

指導教授: 廖世偉 博士

Advisor: Shih-Wei Liao, Ph.D.

中華民國 115 年 6 月

June 2026

doi:10.6342/NTU202601059

國立臺灣大學碩士學位論文

口試委員會審定書

應用於邊緣運算環境之低延遲 Wasm 執行時期分層編譯架構

A Tiered Compilation Architecture for Low-Latency Wasm Runtimes in Edge Computing Environments

本論文係李祥宇君（R12922137）在國立臺灣大學資訊工程研究所完成之碩士學位論文，於民國 115 年 6 月 10 日承下列考試委員審查通過及口試及格，特此證明

口試委員：_____

（指導教授）

_____ _____

_____ _____

_____ _____

_____ _____

所　　長：_____

doi:10.6342/NTU202601059

Acknowledgements

首先感謝廖士偉老師與黃敬群老師在研究過程中的指導。兩位老師在研究方向、問題釐清與論文撰寫上給予我許多建議，使我能逐步整理想法並完成這篇論文。

也感謝實驗室同學們在這段時間的討論與交流。無論是研究上的問題、實作上的細節，或是日常的分享，都讓做研究的過程變得更有趣，也讓我在遇到困難時能持續往前推進。

最後感謝我的父母一路以來的支持。謝謝你們在生活與精神上的陪伴，讓我能安心完成研究與學業。

doi:10.6342/NTU202601059

摘要

WebAssembly 已廣泛部署於邊緣運算與無伺服器運算環境，這類場景對於冷啟動延遲特別敏感，因為其直接影響使用者感受到的回應速度。WasmEdge 為當前主流的伺服器端 WebAssembly 執行時期，目前僅以 LLVM 作為其唯一的 AOT 與 JIT 編譯器。儘管 LLVM 能產生高度最佳化的機器碼，其編譯流程具有顯著的延遲與記憶體開銷，並不適合資源受限的邊緣裝置或生命週期短的函式。WasmEdge 因此存在結構性的冷啟動瓶頸：每次啟動皆須先承擔完整的 LLVM 編譯成本，方能執行任何程式碼。

本論文提出一套適用於 WasmEdge 的分層編譯架構，目的在於將啟動延遲與最終程式碼品質解耦。第一層為 baseline JIT 編譯層 (Tier-1)，於模組載入時以低成本將所有函式編譯為原生程式碼，立即提供執行能力。第二層為最佳化編譯層 (Tier-2)，重用 WasmEdge 既有的 LLVM 前端，透過合成 mini-module、三類 ABI thunk 橋接機制，以及納入靜態熱點呼叫對象的批次組合策略，選擇性地重新編譯熱點函式。第三項為在堆疊替換 (On-Stack Replacement, OSR) 機制，於最外層迴圈反向邊插入監測點，將正在執行中的 tier-1 框架於迴圈執行過程中遷移至 tier-2，解決函式入口替換在「單次呼叫者」場景下無法加速的問題。

本架構於 WasmEdge 中實作，並以強化後的 sightglass 測試套件進行評估。在 39 個分析測試項目上，本架構將編譯時間相較於完整模組 LLVM-JIT 縮短了

doi:10.6342/NTU202601059

幾何平均 11.1 倍，同時於穩態吞吐量上仍保留了約 86% 的 LLVM-JIT 表現。端到端執行時間因此平均縮減 1.73 倍，於最大型工作負載上更展現出數量級的改進：tinygo-json 達 21.9 倍、ONNX 分類器達 14.1 倍、三個 SpiderMonkey 測試項目皆達約 10 倍。OSR 機制亦於單次呼叫者場景中彌補殘餘差距—將 shootout-ctype 由 0.37 倍提升至 0.88 倍、blind-sig 由 0.40 倍提升至 0.80 倍（相對於 LLVM-JIT）。

關鍵字： WebAssembly、及時編譯、分層編譯、在堆疊替換、邊緣運算、WasmEdge

doi:10.6342/NTU202601059

Abstract

WebAssembly is increasingly deployed in edge computing and serverless environments, where cold-start latency directly determines user-perceived responsiveness. WasmEdge, a leading server-side WebAssembly runtime, relies solely on LLVM for both ahead-of-time (AOT) and just-in-time (JIT) compilation. Although LLVM produces highly optimized machine code, its compilation pipeline incurs compile-time and memory overheads that are unsuitable for resource-constrained edge devices and short-lived functions. Every invocation pays the full LLVM compile cost before any code executes.

This thesis presents a tiered compilation architecture for WasmEdge that decouples cold-start latency from peak code quality. A baseline JIT tier compiles every function eagerly at module load time. An optimizing tier reuses the existing WasmEdge LLVM frontend to selectively recompile hot functions through mini-module synthesis, three ABI bridge thunks, and a batch composition policy that admits statically-hot direct callees. An on-stack replacement (OSR) mechanism instruments outermost-loop back-edges in

doi:10.6342/NTU202601059

baseline-tier code and migrates currently-running frames into the optimizing tier mid-execution, addressing the one-shot-caller scenario in which function-entry promotion alone delivers no speedup.

The architecture is implemented in WasmEdge and evaluated on a strengthened variant of the sightglass benchmark suite. Across 39 analyzed kernels, the architecture cuts compile time by a geometric-mean factor of 11.1× over whole-module LLVM JIT while preserving 86% of LLVM-JIT steady-state throughput. End-to-end wall-clock time improves by 1.73× on average, with order-of-magnitude wins on the largest workloads: 21.9× on `tinygo-json`, 14.1× on the ONNX classifier, and approximately 10× on each of the three SpiderMonkey kernels. On-stack replacement closes the residual gap on one-shot-caller workloads, raising `shootout-ctype` from 0.37× to 0.88× and `blind-sig` from 0.40× to 0.80× relative to LLVM JIT.

Keywords: WebAssembly, just-in-time compilation, tiered compilation, on-stack replacement, edge computing, WasmEdge

doi:10.6342/NTU202601059

Contents

doi:10.6342/NTU202601059

doi:10.6342/NTU202601059

doi:10.6342/NTU202601059

doi:10.6342/NTU202601059

List of Figures

doi:10.6342/NTU202601059

List of Tables

doi:10.6342/NTU202601059

doi:10.6342/NTU202601059

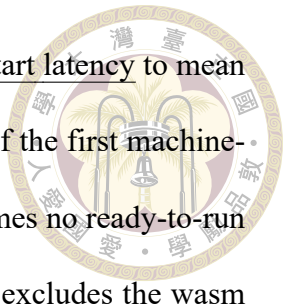
Chapter 1 Introduction

1.1 Motivation

Edge computing has shifted general-purpose computation from centralized data centers toward devices and points-of-presence located close to data sources and end users. Two pressures shape the workloads that run at the edge. The first is a tight per-request response-time budget; tens of milliseconds are routinely required for interactive content delivery, real-time inference, and request-handling at the network edge. The second is a tight per-host resource budget. Edge nodes are typically provisioned with substantially less CPU, memory, and energy headroom than data-center servers. The workload mix is dominated by short-lived, stateless functions that frequently start and exit.

WebAssembly (Wasm) [18, 26] has emerged as the deployment vehicle for these workloads. Its compact binary format, language-agnostic toolchain, sandboxed execution model, and predictable resource semantics make it well suited to multi-tenant environments in which untrusted code must be loaded, executed, and torn down at high frequency. Production systems including Fastly Compute [12], Cloudflare Workers [10], and Fermyon Spin [13] embed Wasm runtimes for exactly this reason. The class of workloads these systems target is one for which cold-start latency is as critical as steady-state throughput [15]: a serverless function invoked once and discarded pays the full cost of compila-

doi:10.6342/NTU202601059



tion before producing any user-visible output. This thesis uses cold-start latency to mean the time from request arrival at the runtime to the start of execution of the first machine-code instruction of the requested wasm function. The definition assumes no ready-to-run compiled instance of the module is already resident on the host, and excludes the wasm function's own execution time, which is reported separately as steady-state work.

These cloud-native runtimes share a common deployment model. Each incoming request is dispatched to a freshly-instantiated Wasm sandbox that loads, executes, and tears down within the request handler's lifetime. The model enables strong tenant isolation without the kernel-level overhead of containers or virtual machines, and the language-agnostic toolchain accepts modules compiled from C, Rust, Go, and other source languages without runtime-specific build adaptations. The compile-time profile of the underlying runtime therefore bounds the response-time floor that the platform can expose to the user. Production deployments hide part of this cost with ahead-of-time compilation caches and pre-warmed sandbox pools. Ahead-of-time compilation amortizes only across repeated reuse of a module known in advance. It does not reduce the latency of the first invocation of a previously-unseen module. Diverse, first-seen modules are the common case on multi-tenant edge and serverless nodes. Ahead-of-time compilation does not help with them. The per-tenant cold path therefore remains. The structural limitation is that single-tier optimizing compilation forces every invocation through the same compile pipeline, regardless of whether the request will execute the module for one microsecond or for one minute.

WasmEdge [25] is an open-source Wasm runtime developed under the Cloud Native Computing Foundation and adopted in production by ByteDance, Microsoft, Sony, and others. Its compile path uses LLVM end-to-end for both ahead-of-time (AOT) and just-in-

doi:10.6342/NTU202601059

time (JIT) compilation. LLVM produces highly optimized machine code. Its optimization pipeline includes mid-level inlining, loop transformations, vectorization, and global register allocation. The result is throughput close to native C compilation on computationally intensive workloads. These optimizations come at a cost. LLVM's compile-time profile scales poorly with module size, and its memory footprint during compilation is substantial. On the resource-constrained hosts and short-lived functions that characterize edge deployments, paying this cost on every invocation forces a structural trade-off: the run-time cannot deliver a low cold-start latency without sacrificing peak code quality.

A second observation complicates the single-tier design. Within a typical Wasm module, the distribution of execution time across functions is highly skewed: a small fraction of functions accounts for the bulk of run-time, while the remainder execute infrequently or not at all. Compiling every function with LLVM-grade optimization spends most of the compile budget on code that will not run hot. Cold or rarely-executed code receives optimization it does not need, and the compile time imposed on hot code delays the program from reaching its hot path.

This pattern is well-established in the design of high-performance language runtimes. Modern JavaScript engines—V8 [17], JavaScriptCore [21], SpiderMonkey [7]—all employ tiered compilation. A fast baseline tier produces native code for every function with minimal optimization. One or more optimizing tiers then selectively recompile hot code with progressively heavier optimization, guided by run-time profile information. The baseline tier eliminates the cold-start gap; the optimizing tier delivers steady-state performance comparable to a single-tier optimizing compiler. Wasm runtimes built atop these engines, including V8's Liftoff and JavaScriptCore's BBQ tier, follow the same architecture [17, 21]. WasmEdge does not. The runtime currently lacks a baseline JIT, lacks a

doi:10.6342/NTU202601059

profile-driven promotion mechanism, and lacks a mid-execution code-replacement pathway. The cold-start bottleneck observed in production is a direct consequence of this architectural gap.

1.2 Problem Statement

This thesis addresses the absence of a tiered compilation infrastructure in WasmEdge. The goal is to introduce an architecture that decouples cold-start latency from peak code quality, while preserving the steady-state performance that LLVM-compiled code achieves on hot workloads. Three concrete sub-problems constitute this goal.

Baseline tier. The runtime requires a compiler that produces native code for every wasm function at low cost at module load time. The compiler must integrate with WasmEdge's existing runtime contracts—linear memory layout, table dispatch, host-call boundaries, trap handling, and the function-instance store. It must lower the full set of MVP WebAssembly instructions plus the reference-type, bulk-memory, and table extensions used by sightglass and real workloads. It must produce code of sufficient quality that the resulting steady-state performance is acceptable until tier-2 compilation completes. A lightweight SSA-based code generator with linear-scan register allocation is an established design point that meets these constraints.

Optimizing tier. The runtime requires a path that recompiles hot wasm functions with LLVM-grade optimization, but only after profile information has identified them as worth the compile cost. Reimplementing WasmEdge's existing wasm-to-LLVM lowering is undesirable: the LLVM frontend is mature, well-tested, and already encodes the module-

doi:10.6342/NTU202601059

level lowering logic that selective recompilation also requires. Two challenges arise. The first is to drive the existing LLVM frontend with input that contains only the hot functions, while preserving the call-graph structure on which the frontend's call lowering depends. The second is to bridge the resulting LLVM-compiled code into the baseline tier's calling convention so that mixed-tier dispatch is correct. The promotion mechanism must run asynchronously to user code so that LLVM's compile latency is hidden from the request-handling path, and the published code must replace the baseline-tier entry atomically with respect to concurrent callers.

In-flight migration. Function-entry promotion alone is insufficient for an important class of workloads. Consider a function called once that spends the bulk of its execution inside a single long-running loop. This is the shape of cryptographic kernels, parsing loops, and many serverless request handlers. The function never re-enters its prologue after the counter saturates, so the entry-table swap delivers no speedup on the in-flight invocation. Closing this gap requires a mechanism that detects when an executing tier-1 frame has crossed a hotness threshold inside a loop, migrates the frame's wasm-level state into a tier-2 continuation entry, and transfers control mid-execution. The mechanism must coexist with function-entry promotion, must impose negligible overhead in the steady state once it has fired, and must preserve wasm semantics across the transition.

Each sub-problem presents system-level challenges that go beyond the compilation algorithm itself. The list includes the choice of profile representation and storage, concurrent publication of native code, lifetime management of code caches that outlive their compiling thread, and the interaction between tier-1 and tier-2 calling conventions on the indirect-call paths that Wasm modules use heavily. The remainder of this thesis presents

doi:10.6342/NTU202601059

a design that resolves each of these constraints, an implementation in WasmEdge, and an evaluation of the resulting system.

1.3 Contributions

This thesis makes the following contributions.

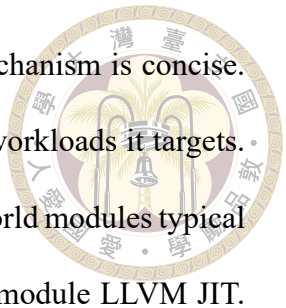
1. **A baseline JIT tier (Tier-1) for WasmEdge.** The proposed tier integrates a lightweight Sea-of-Nodes JIT backend as a compile-every-function pathway invoked at module instantiation. The backend runs its standard optimization pipeline. Profile counters embedded in the compiled bodies drive subsequent tier-2 promotion.

2. **A selective LLVM-frontend recompilation tier (Tier-2).** The proposed tier reuses the existing WasmEdge LLVM frontend by synthesizing per-batch mini-modules that contain only hot functions. A tier-up heuristic selects which functions enter each batch. Three ABI thunks bridge the two tiers' calling conventions.

3. **An on-stack replacement (OSR) mechanism for tier-1 to tier-2 migration.** The proposed mechanism instruments outermost-loop back-edges in tier-1 code to detect hot loops. The mechanism then migrates the executing tier-1 frame into a tier-2 continuation entry mid-loop. The migration closes the gap on workloads where function-entry promotion alone delivers no speedup.

4. **Empirical evaluation.** The thesis evaluates the architecture on the sightglass benchmark suite, with whole-module LLVM JIT as the steady-state-throughput baseline. The chapter thoroughly analyzes the results per workload shape and isolates the contribution of each tier-2 mechanism.

doi:10.6342/NTU202601059



The architecture is implemented in WasmEdge. The tier-up mechanism is concise. The evaluation shows it is also effective on the edge and serverless workloads it targets. Cold-start latency drops by up to one order of magnitude on the real-world modules typical of these deployments. Steady-state throughput stays close to whole-module LLVM JIT. To the best of the author's knowledge, this combination is not documented in the browser-engine or server-side WebAssembly literature in this exact form.

1.4 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 reviews the WebAssembly execution model, the WasmEdge runtime, the structural cost of LLVM compilation, the dstogov/ir library, the principles of tiered compilation, and prior work on tiered compilation in JavaScript and Wasm runtimes, on-stack replacement, and lightweight JIT libraries. Chapter 3 presents the design of the proposed system: the architecture overview, the runtime context and calling conventions, the tier-1 baseline JIT, the tier-2 selective recompilation pipeline, and the on-stack replacement mechanism. Chapter 4 reports the evaluation: methodology, results across the three components, cold-start latency, and sensitivity sweeps. Chapter 5 concludes the thesis and discusses limitations and future work.

doi:10.6342/NTU202601059

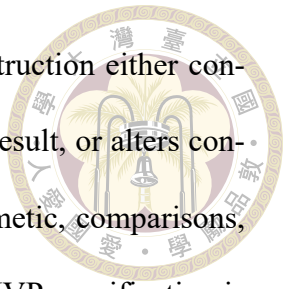
Chapter 2 Background and Related Work

This chapter provides the technical background on which the rest of the thesis builds. Section 2.1 reviews the WebAssembly execution model. Section 2.2 describes the WasmEdge runtime and its existing compile paths. Section 2.3 characterizes the cost profile of LLVM-based compilation, which motivates the tiered architecture. Section 2.4 introduces the lightweight Sea-of-Nodes JIT library used as the tier-1 code generator. Section 2.5 summarizes the principles of tiered compilation. Section 2.6 surveys related work.

2.1 The WebAssembly Execution Model

WebAssembly is a low-level bytecode format designed for portable, sandboxed execution [18, 26]. A WebAssembly module is a binary container divided into typed sections: function signatures, function bodies, imports, exports, tables, linear memories, globals, and an initialization section for elements and data. The binary is validated in a single pass before any instruction executes. Validation enforces type safety on the operand stack, valid branch-target depths, and in-range indices for the function, local, global, table, and memory spaces.

doi:10.6342/NTU202601059



WebAssembly defines a stack-based virtual machine. Each instruction either consumes typed operands from the operand stack and produces a typed result, or alters control flow. The instruction set covers integer and floating-point arithmetic, comparisons, memory load and store, function calls, and structured control. The MVP specification is augmented by feature proposals that this thesis targets: reference types, bulk-memory operations, and table operations. SIMD (`v128`) is not targeted in the current implementation; see Section 5.2.

A module's defined functions, together with its imported functions, form an indexed function space. Direct calls reference these functions by index. Indirect calls reference a function pointer through a typed table, and the runtime checks the table entry's signature against the call site's expected signature at run time. Tables support host-controlled writes and runtime growth.

Linear memory is a byte-addressable buffer that grows in fixed-size pages. Load and store instructions specify a base address and a static offset. The runtime is required to bounds-check every access and to trap on a violation. Linear memory is not shared with the host process address space. This isolation is the foundation of WebAssembly's sandboxing guarantee.

Control flow is structured rather than free-form. Blocks, loops, and conditionals open scopes that can be exited only through labelled branch instructions, and every branch target is a syntactic structure visible from the branch instruction. The restriction simplifies validation and gives optimizing compilers a regular control-flow shape to operate on.

A module instance is a runtime materialization of a module against a specific store, with concrete function instances, table instances, memory instances, and global instances

doi:10.6342/NTU202601059

bound. Multiple module instances of the same module may coexist with independent state. Host functions—functions implemented by the embedder rather than by the wasm bytecode—are imported into the module instance through the same indexed function space as defined wasm functions. Call sites therefore do not distinguish between defined functions and host functions.

2.2 The WasmEdge Runtime

WasmEdge [25] is an open-source WebAssembly runtime hosted by the Cloud Native Computing Foundation. The runtime targets cloud-native and edge deployments and is used in production by ByteDance, Microsoft, Sony, and others. Its codebase implements the full WebAssembly MVP specification, every standardized extension that the proposals working group has accepted, and several WASI standards.

WasmEdge provides three execution paths for a parsed and validated module. The interpreter walks the wasm bytecode directly, dispatching one instruction per loop iteration through a switch over the opcode. The interpreter is the simplest of the three paths and the slowest. It exists for correctness reference, for environments in which JIT-compiled code is unwelcome, and for kernels and instructions for which the compiled paths have not been implemented.

The two compiled paths share a common frontend. WasmEdge's LLVM frontend translates wasm bytecode to LLVM intermediate representation. It lowers wasm operations to typed LLVM operations, wasm structured control to LLVM basic blocks, and wasm linear memory accesses to LLVM loads and stores with explicit bounds-check guards. The frontend is reused by both compiled paths and is therefore the runtime's single source

doi:10.6342/NTU202601059

of wasm semantic lowering.

The ahead-of-time path runs the frontend at build time. The resulting LLVM module is compiled to a shared object that the runtime loads back at module instantiation. The shared object bypasses the frontend entirely on the loading thread, so cold start is dominated by file I/O and dynamic linking rather than by compilation. The path is intended for production deployments in which the wasm module is known in advance and can be compiled once.

The just-in-time path runs the same frontend at module instantiation. The resulting LLVM module is passed through LLVM's optimization pipeline and its just-in-time machine code emitter, and the produced function pointers are bound into the module instance. The path produces code of comparable quality to the ahead-of-time path. The path also pays the full LLVM compile cost on every cold start. That cost is the bottleneck this thesis addresses.

Both compiled paths optimize every function uniformly. Neither observes a runtime profile, so neither distinguishes a hot function from a cold one. The ahead-of-time path absorbs this cost offline, where uniform optimization is acceptable. The just-in-time path pays it at module instantiation, where optimizing functions the input never exercises wastes the user's time.

All three paths share the same module-instance representation and the same host-call dispatch boundary. A wasm-to-host call lands in a stub that marshals arguments from the wasm convention into the host's native convention and back. A host-to-wasm call goes through the runtime's executor to ensure that the trap state and stack guards are correctly maintained. The shared boundary is what allows this thesis to introduce a new compiled

doi:10.6342/NTU202601059

path—the tier-1 baseline JIT and selective tier-2 promotion—without changes to the executor's external interfaces.

2.3 LLVM Compilation Cost

LLVM [20] is a mature, production-grade optimizing compiler infrastructure. Its optimization pipeline at `O2` and above is comparable in coverage and aggressiveness to the optimizing tiers of major language compilers. The pipeline is also expensive. The cost has three sources.

The first source is the size of the pipeline itself. LLVM at `O2` runs dozens of analysis and transformation passes over the intermediate representation, including inlining, loop transformations, vectorization, and global value numbering. The passes operate on a single intermediate representation that is repeatedly rewritten as each transformation fires. Each pass adds constant overhead even on inputs it does not transform, because the analysis it depends on must still execute.

The second source is the size of the input. Compile time scales roughly linearly with the number of LLVM instructions, but specific passes scale superlinearly. Inlining and global value numbering are common offenders: their cost grows with both the number of call sites and the size of inlined bodies. A WebAssembly module with thousands of functions and tens of thousands of LLVM instructions per function exposes both kinds of growth.

The third source is memory footprint. The optimization pipeline holds the entire module in memory and rewrites it in place. Analyses build derived data structures that may rival the module itself in size. The footprint matters because edge deployments routinely

doi:10.6342/NTU202601059

run on hosts provisioned with one or two gigabytes of memory across the entire workload, of which compilation may consume a noticeable fraction.

The three sources combine to produce a compile-cost profile that is acceptable when paid once at build time on the AOT path, but is structurally unsuited to a cold-start path. The tiered architecture in Chapter 3 reduces the up-front cost by routing the cold path through a much smaller code generator and reserving LLVM for the small subset of functions that a runtime profile shows to be hot.

2.4 A Lightweight Sea-of-Nodes JIT Library

This thesis adopts dstogov/ir [23], an open-source JIT compilation framework developed by Dmitry Stogov, the lead developer of the Zend Engine and the author of the Zend JIT shipped in PHP 8.0 and later. The library is a productized extraction of the same code generator that powers PHP's JIT path, designed from the outset to fit the compile-time budget of a JIT compiler rather than the compile-time budget of an ahead-of-time toolchain: a fixed linear-time pass pipeline, no LLVM-style pass-manager abstraction, and a linear-scan register allocator on SSA. These engineering choices yield compile-time profiles that scale linearly with function size and remain bounded under adversarial input, which is the property a baseline JIT requires on the cold path. The library's intermediate representation is Sea-of-Nodes in the sense of Click and Paleczny [9]: data and control dependencies are unified in a single graph of typed nodes, and most nodes are "floating"—not pinned to any basic block—until a late scheduling pass places them. The same representation underlies HotSpot's C2 server compiler, V8's TurboFan, and Graal. Because the IR is SSA by construction, the library exposes a single-pass builder API that emits

doi:10.6342/NTU202601059

nodes one at a time and never needs a separate SSA-construction phase.

The optimization sequence is short and fixed. A folding engine runs during IR construction, performing constant folding, copy propagation, algebraic simplification, and common-subexpression elimination via value-numbered node deduplication. A sparse conditional-constant-propagation pass [23] folds branches and arithmetic on values that become constant under the SSA control flow. A global code motion pass after Click [8] builds the CFG skeleton and pins each floating node to its best basic block, hoisting loop-invariant computations and sinking expressions out of hot blocks. A local scheduling pass then orders instructions within each block by topological sort. Target instruction selection performs Max-Munch tree pattern matching to combine adjacent nodes into single host instructions. A linear-scan register allocator on SSA assigns values to physical registers. The sequence is fixed: there is no pass manager, no opt-level abstraction, and no extension point for analyses that re-traverse the IR.

The fixed sequence is the source of the library's two attractive properties for a baseline JIT. Compile latency is bounded and predictable, because the cost of every pass scales linearly with the function size. The library's source code is small enough to vendor into the runtime and modify when a behavioural change is required. Both properties are essential to the tier-1 design described in Chapter 3: the runtime needs predictable compile cost on the cold path, and the thesis relies on the freedom to extend the library with backend changes that the project might not accept upstream.

The trade-off is code quality. The library does not perform inlining, does not vectorize, and does not run iterative dataflow analyses beyond the fixed sequence. Its output is competitive with a non-optimizing reference compiler and an order of magnitude below

doi:10.6342/NTU202601059

LLVM at O2. This gap is precisely the gap the tier-2 path closes for hot functions.

2.5 Tiered Compilation Principles

Tiered compilation uses two or more compilers of different cost-quality profiles to compile the same program. Profile information directs which compiler runs on which function. The strategy is established in production language runtimes for one consistent reason. The distribution of execution time across the functions of a typical program is highly skewed. A single optimizing compiler spends most of its compile budget on functions that contribute little to run-time. The pattern generalizes beyond managed-language runtimes. Chen et al. [6] apply the same interpreter-baseline-optimizer layering to RISC-V instruction-set simulation.

A baseline tier produces native code at low compile cost. The output is correct and reasonably fast. The output does not benefit from heavy optimization. Common baseline-tier designs use a template-based code generator or a lightweight SSA-based code generator. The baseline tier eliminates the cold-start gap. The runtime can begin executing the program as soon as parsing and the baseline compile complete.

An optimizing tier produces code of higher quality at higher compile cost. The runtime engages the optimizing tier selectively. The runtime recompiles only the functions identified as worth the cost. Profile information collected from executing baseline-tier code drives the identification. The optimizing tier delivers steady-state performance comparable to a single-tier optimizing compiler. The benefit accrues only to code that runs hot.

Profile collection identifies hot functions through one of several mechanisms. Some

doi:10.6342/NTU202601059

runtimes embed counters into baseline-tier code. Some runtimes use interpreter-side counters. Some runtimes sample the program counter. Some runtimes combine counter increments with age-based decay. The choice of mechanism is a design decision that varies across runtimes. The shared goal is to mark functions that cross a hotness threshold.

Promotion takes one of two forms. Function-entry promotion redirects future calls of a hot function to its optimized body. The redirection mechanism varies: dispatch-table swap, call-site patching, or inline-cache patching. The currently-executing frame, if any, continues to run baseline-tier code, because the frame has already passed through the prologue.

On-stack replacement migrates an executing frame mid-function into an optimized continuation entry. The state-transfer mechanism varies: stack maps in some runtimes, structured local-set transfers in others. On-stack replacement matters when a function is called once and spends nearly all its execution inside a loop. Function-entry promotion delivers no speedup on such workloads.

The optimizing tier's compile cost must not block user code. Production tiered runtimes therefore run optimizing-tier compilation asynchronously on a worker thread. The user thread executes baseline-tier code, accumulates profile information, and notifies the worker on threshold crossings. The worker compiles and publishes the result. Publication must be safe under concurrent access. The publication mechanism varies: atomic table write, code-cache update with read-copy-update, or call-site patching.

The architecture described in Chapter 3 instantiates these principles for WasmEdge.

doi:10.6342/NTU202601059

2.6 Related Work

The proposed architecture sits at the intersection of three established lines of work: tiered compilation in browser-engine WebAssembly pipelines, low-latency baseline compilation in server-side WebAssembly runtimes, and on-stack replacement as a runtime migration mechanism. This section surveys each, then states how the proposed design relates to them.

2.6.1 Browser-engine WebAssembly tiering

The three major browser-engine WebAssembly pipelines all use multi-tier execution.

V8 pairs Liftoff with TurboFan [17]. Liftoff is a single-pass baseline compiler that maps wasm operations directly to machine registers without constructing an intermediate representation, emitting code at a documented rate of tens of megabytes per second. TurboFan is V8's multi-pass optimizing compiler, invoked on hot functions identified by per-function execution counters. V8's documentation states that the pipeline does not perform on-stack replacement for WebAssembly: hot functions tier up at the next call, and any in-flight invocation runs to completion in Liftoff code. V8's more recent dynamic-tiering work [16] addresses the orthogonal cost of whole-module optimizing recompilation by deferring TurboFan to functions that prove hot at run time, rather than compiling the entire module eagerly.

SpiderMonkey pairs Baseline with Ion [7]. Baseline is the low-latency tier, reported as ten to fifteen times faster than Ion at compile time. Ion is the optimizing tier. Firefox's documented configuration historically performed eager whole-module Ion compila-

doi:10.6342/NTU202601059

tion after Baseline produced its initial output; subsequent work moved toward function-granularity Ion compilation triggered by warm-up counters.

JavaScriptCore now operates a three-tier WebAssembly pipeline [21]. The lowest tier is an interpreter (IPInt) that begins execution as soon as the module is parsed. BBQ (Build Bytecode Quickly) is the baseline JIT, documented as approximately four times faster at compile time than the optimizing tier. OMG (Optimized Machine-code Generator) is the optimizing JIT, invoked on hot functions. WebKit publishes call-site hot-patching as its function-promotion mechanism, and recent work has added interpreter-to-BBQ on-stack replacement; BBQ-to-OMG OSR is not documented in the public material.

The proposed design shares the startup-versus-throughput trade-off with all three pipelines and the selective hot-function promotion policy with V8's dynamic-tiering work. It differs in three respects. First, the baseline tier is built on a vendored Sea-of-Nodes code generator rather than on a template or single-pass emitter; the trade-off is discussed in Section 2.4. Second, the optimizing tier reuses an existing wasm-to-LLVM frontend rather than a custom optimizing backend, which requires the mini-module synthesis and ABI bridge mechanism described in Chapter 3. Third, the architecture targets a server-side WebAssembly runtime; the baseline tier's compile-latency budget is tighter than Liftoff's or Baseline's, and the optimizing tier's compile-cost profile is heavier than TurboFan's, Ion's, or OMG's.

2.6.2 Server-side WebAssembly runtimes

Wasmtime [5] is the closest server-side analogue to WasmEdge. Its default backend is Cranelift [3], an optimizing code generator designed to balance compile speed and code

doi:10.6342/NTU202601059

quality. The Wasmtime baseline-compilation RFC [2] introduces Winch (WebAssembly Intentionally Non-optimizing Compiler and Host) as a low-latency baseline that emits machine code directly from wasm bytecode in a single forward pass, using Cranelift's instruction emitters as a macro-assembler. Wasmtime's documentation states that the runtime does not currently support automatic tier-up from Winch to Cranelift: a module is compiled entirely by one backend or the other at the user's choice.

The proposed design differs from Wasmtime's current state in two ways. First, the architecture provides an automatic tier-up pathway: tier-2 promotion is driven by execution profile rather than a build-time strategy choice. Second, the design introduces on-stack replacement as part of the same tier-up pathway, addressing the workload shape that function-entry promotion alone cannot accelerate.

2.6.3 On-stack replacement

On-stack replacement was introduced in the Self programming language's adaptive compilation pipeline as a mechanism for transferring an executing activation from one compiler's output into another's [19]. The Self work used the mechanism in both directions: to enter optimized code from baseline code, and to recover from speculative optimizations that proved invalid at run time. Fink and Qian formalized OSR as the mechanism behind adaptive recompilation in the Jikes RVM [14], with stack maps generated by the baseline compiler describing the location of live variables at every potential transfer point and the optimizing compiler synthesizing prologue code that unpacks the stack map and reconstructs its own frame layout. D'Elia and Demetrescu provide a contemporary taxonomy of OSR mechanisms [11], distinguishing the transfer semantics (the correctness contract on values that must survive the transition) from the runtime engineering choices

doi:10.6342/NTU202601059

(stack-map representation, transfer-point granularity, continuation synthesis).

Modern production virtual machines treat OSR as a standard part of the tiering pathway. The Java HotSpot virtual machine has supported OSR for over two decades [22]. A back-edge counter inside each compiled method triggers an OSR compile once the counter crosses a threshold. The OSR compile produces a second version of the method whose entry point is the loop header. The runtime then copies live local-variable state from the running frame into the new frame at the transfer point. The .NET CoreCLR runtime added the same mechanism in .NET 7, released in 2022 [24]. The .NET release notes give the same motivation that drives this thesis: a method called once that spends most of its time inside a loop gets no benefit from function-entry promotion alone.

A separate family of runtimes uses trace compilation rather than OSR to enter optimized code from a running unoptimized frame. Tracing JIT compilers such as PyPy [1] and LuaJIT start compiling when a back-edge counter crosses a threshold. The compiler records the actual execution path taken through the loop body. The recorded trace is then compiled as a code fragment whose entry point is the loop header. The interpreter can call into the trace directly on the next iteration, with no separate state-transfer step. Trace compilation is the main alternative to OSR-based tier-up in the literature. The approach is incompatible with this thesis's architecture: reusing the existing WasmEdge LLVM frontend requires a function-granularity compilation unit, not a trace-granularity one.

On-stack replacement also has a reverse direction. OSR-exit moves an executing optimized-tier frame back into a lower tier when an assumption the optimizer made turns out to be wrong at run time [19]. HotSpot's "uncommon trap" mechanism and JavaScriptCore's FTL OSR-exit both implement this direction. The OSR-exit direction lets the op-

doi:10.6342/NTU202601059

timizing tier make assumptions about the program that the baseline tier does not. This thesis implements only the OSR-entry direction. The absence of OSR-exit is discussed as a limitation in Chapter 5.

The OSR mechanism described in Chapter 3 follows the HotSpot and .NET-7 lineage outlined above and sits at the simpler end of the spectrum that D'Elia and Demetrescu describe. The transfer points are restricted to outermost-loop back-edges. The live state at each transfer point is exactly the set of wasm-level locals—a syntactically delimited and typed set rather than the register-and-spill location set that HotSpot-style stack maps must describe. The continuation is synthesized as a fresh wasm function whose entry point is the loop header. The simplification is made possible by WebAssembly's structured control flow and its explicit local-variable model. The price is a conservative filter that rejects loops whose surrounding control flow cannot be safely re-entered mid-execution; the rejection is discussed in Section 3.4.3.

doi:10.6342/NTU202601059

Chapter 3 Design

This chapter presents the design of the proposed tiered compilation architecture. Section 3.1 introduces the three components—tier-1 baseline JIT, tier-2 selective recompilation, and on-stack replacement—and the pipeline that connects them. Section 3.2 describes the tier-1 baseline JIT, its runtime context, and its embedded profile counters. Section 3.3 describes tier-2 selective recompilation and the calling-convention bridge to tier-1. Section 3.4 describes on-stack replacement. The presentation focuses on the design choices that distinguish each mechanism. Pseudocode and algorithm blocks supplement the prose where a mechanism's structure is non-obvious.

3.1 Architecture Overview

The proposed architecture has three components. A lightweight baseline JIT compiles every wasm function at module instantiation. The resulting tier-1 native code runs on the user thread with embedded profile counters. A background worker thread selectively recompiles hot functions through the existing WasmEdge LLVM frontend.

Two promotion paths target the two invocation shapes that matter. The first, function-entry tier-2 promotion, redirects future calls to a hot function into LLVM-compiled code by atomically replacing the function's entry in the dispatch table. The second, on-stack

doi:10.6342/NTU202601059

replacement, migrates a currently-executing tier-1 frame into LLVM-compiled code mid-loop, so the in-flight invocation also benefits from optimization. Figure 3.1 summarizes the architecture.

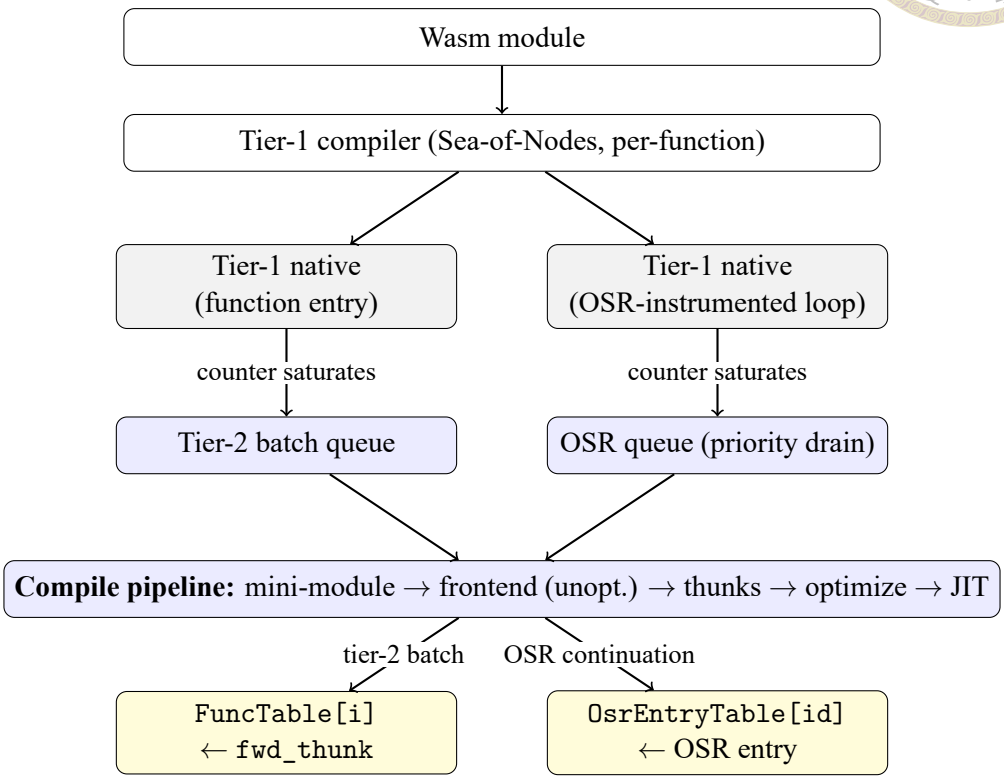


Figure 3.1: Architecture of the proposed tiered compilation system. The tier-1 compiler (top) lowers every defined function at module instantiation. Tier-1 native code (gray) executes on the user thread and embeds two classes of profile counter. On counter saturation, one of two notifications fires and enqueues a compile request on the tier-2 worker (blue), which drains the OSR queue first and executes a single shared compile pipeline. The result publishes atomically into one of two shared slots (yellow): the `FuncTable` for function-entry promotion, the `OsrEntryTable` for OSR.

Figure 3.1 walks through the five stages of the pipeline. The first three execute on the user thread at module load and during execution; the last two execute asynchronously on the tier-2 worker thread when a profile counter saturates.

The proposed pipeline coexists with the existing whole-module LLVM AOT path. A module that has already been AOT-compiled to a shared library bypasses tier-1 entirely. The two paths suit different cases. Ahead-of-time compilation serves modules known in advance and reused often enough to pay the compile cost once. The tiered path serves

doi:10.6342/NTU202601059

first-seen, short-lived modules, where the cost must be paid at request time. The tiered architecture is therefore a strict addition, not a replacement.

3.2 Tier-1 Baseline JIT

The tier-1 baseline JIT compiles every defined wasm function at module instantiation, producing native code at low compile cost. The output of each compilation is placed in the `FuncTable` for use by direct and indirect dispatch. Three design choices distinguish the proposed mechanism: the choice of a lightweight Sea-of-Nodes code generator, a single-pass wasm-to-IR translation strategy with selective PHI emission, and the embedding of profile counters directly into the compiled bodies. The lowering covers the full MVP WebAssembly instruction set plus the reference-type, bulk-memory, and table extensions; SIMD (`v128`) instructions are not lowered in the current implementation, so a function that uses them fails tier-1 compilation.

3.2.1 Runtime context and calling convention

Every tier-1-compiled function receives a single pointer-typed argument—a `JitExecEnv` instance—through which it reads all module-level state. The single-struct design serves two purposes: it removes any need for the tier-1 hot path to traverse the executor's data structures, and it provides a stable lowering target for the tier-2 ABI bridge thunks (Section 3.3.1). Table 3.1 summarizes the three field groups.

The tier-2 worker thread writes to only one field of `JitExecEnv`: the `OsrEntryTable` pointer. All other fields are read or written exclusively by the user thread.

doi:10.6342/NTU202601059

Group	Purpose
Lookup tables	Pointers to long-lived runtime state: the per-module function pointer array (`FuncTable`), the linear-memory data pointer, the globals array, and the indirect-call shadow dispatch table.
Helper addresses	Addresses of runtime helper functions invoked by tier-1 code: host-call dispatch, indirect-call dispatch, memory-grow and memory-size queries, and the two profile-saturation notification helpers.
Promotion state	Pointers to long-lived storage that drives tier-2 promotion: the per-function call counter array, the per-loop back-edge counter array, the OSR entry pointer table (`OsrEntryTable`), and a per-thread buffer used to serialize wasm locals across the OSR transition.

Table 3.1: The three field groups of `JitExecEnv`.

Every tier-1-compiled wasm function exposes a uniform two-argument signature: an environment pointer (the `JitExecEnv`) and an args buffer of 64-bit-aligned slots, one per wasm parameter. Wasm parameter and return types are widened or bit-cast to fit the slot convention. The uniformity reduces tier-1 code generation to a single dispatch shape and lets direct, indirect, and host-call paths share the same trampoline structure.

3.2.2 Lightweight Sea-of-Nodes backend as the code generator

The proposed architecture adopts dstogov/ir [23], the lightweight Sea-of-Nodes JIT library introduced in Section 2.4, as the tier-1 code generator. The library implements a small fixed sequence of optimization passes—on-the-fly folding (with CSE), sparse conditional constant propagation, global code motion, local scheduling, and linear-scan register allocation—behind a simple emitter API that accepts SSA nodes one at a time. The selection over alternatives such as Cranelift [3] or hand-rolled emitters reflects three considerations: SSA-quality output from a single-pass frontend, bounded and predictable compile latency, and a small enough codebase to vendor and modify when the runtime needs a behavioral change in the backend. The library compiles at the function granu-

doi:10.6342/NTU202601059

larity using its standard optimization level. A lower-optimization mode is retained as a debugging aid.

3.2.3 Single-pass wasm-to-IR translation

The translation from wasm bytecode to IR runs as a single forward walk over the function body. The walker simulates the wasm operand stack as a stack of SSA value references, maintains a map from each wasm local index to its current SSA definition, and tracks open structured-control constructs on a label stack. Wasm locals are lowered directly to SSA values rather than to memory cells, which is feasible because the IR is SSA by construction and the local set is known statically at function entry. Most wasm operations lower one-to-one to a single IR node. A few operations face a type-system mismatch between wasm semantics and the backend's typing rules—notably unsigned division and floating-point copysign—and bridge through extern-C helper calls.

One design choice is worth noting. At each loop, the walker performs a pre-pass over the loop body to identify exactly which locals are written within the loop, and emits PHI nodes only for those locals. A naive translation that emitted PHIs for every local would inflate IR size by an order of magnitude on functions with many locals and few-modification loops. Selective PHI emission preserves the SSA values of unmodified locals across iterations, avoiding the inflation.

3.2.4 Profile instrumentation embedded in compiled bodies

The counters that drive tier-2 promotion are emitted directly into the tier-1 compiled bodies rather than collected by external instrumentation. Embedding the counters in the

doi:10.6342/NTU202601059

compiled body has two advantages over an external profile-collection scheme. First, no additional indirection exists on the hot path: a saturated counter short-circuits before the increment, so both a saturated function entry and a saturated back-edge run only three IR operations. Second, the counter access pattern is local to the function being instrumented, so the cache footprint of profile collection scales with the number of currently-executing functions rather than with module size.

Per-function call counter. Each tier-1 function prologue increments a per-function call counter on every entry, gated by an unsigned comparison against the configured threshold. When the increment reaches the threshold, the prologue calls the function-entry tier-up notification helper. The helper saturates the counter to its maximum value to prevent re-firing and enqueues a tier-2 batch compile request. The unsigned comparison is essential: a signed comparison would treat the saturated value as negative and let the gate pass, causing the counter to wrap and re-fire notification on every threshold-th invocation thereafter.

Per-loop back-edge counter. Each outermost-loop back-edge increments a per-loop back-edge counter, with a fixed bound on the number of OSR-eligible loops per function. The counter is gated by the same unsigned-comparison pattern as the call counter, with one additional layer: the back-edge logic also examines a per-loop slot of the `OsrEntryTable` that encodes the OSR state machine. The full three-state encoding is described in Section 3.4.2.

The two thresholds—one for function-entry tier-2 promotion, one for OSR—are independent. A function may be entry-promoted to tier-2 and have OSR continuation entries

doi:10.6342/NTU202601059

published for one or more of its loops. Function-entry promotion and OSR target different invocation shapes and do not interfere.

3.3 Tier-2 Selective Recompilation

The tier-2 path recompiles hot wasm functions with LLVM-grade optimization, but only after profile information has identified them as worth the compile cost. The proposed methodology reuses the existing WasmEdge LLVM frontend by synthesizing per-batch mini-modules and driving the frontend on those mini-modules. The remainder of this section presents the bridging conventions to tier-1, the worker thread, the synthesis idea, the compile-ordering invariant, the role of the bridge thunks, the batch composition algorithm, and the atomic publication step.

3.3.1 Bridging conventions

Every tier-2-compiled wasm function exposes the calling convention emitted by the WasmEdge LLVM frontend: typed parameters in native registers, a typed return, and a leading executor-context pointer. The tier-2 pipeline reuses this frontend—WasmEdge's existing wasm-to-LLVM-IR lowering—rather than introducing a separate lowering, and therefore inherits the frontend's calling convention. The tier-1 and tier-2 calling conventions disagree in three ways relevant to the bridging design: parameter passing (typed registers versus a flat 64-bit slot buffer), context-pointer expectation (executor context versus `JitExecEnv`), and the symbol naming convention used by the LLVM frontend.

A single running module mixes tier-1 and tier-2 native code, producing three classes of cross-tier dispatch. First, tier-1 code calls tier-2 code when the entry-table swap pub-

doi:10.6342/NTU202601059

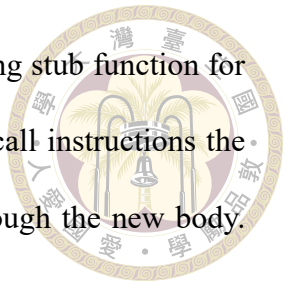
lishes a tier-2 function pointer into the `FuncTable`. Second, tier-2 code calls tier-1 code when a tier-2 batch body invokes a helper that was not part of the batch. Third, LLVM-compiled code (tier-2, OSR, or whole-module LLVM) issues an indirect call whose target may be either tier-1 or tier-2 native code, depending on which functions have been promoted. The architecture handles all three with three small bridge functions called thunks. Table 3.2 summarizes them. Each thunk covers one direction, so neither the tier-1 lowering nor the LLVM frontend has to understand the other's calling convention.

Thunk	Direction	Where it lives	Purpose
`fwd_thunk`	Tier-1 → Tier-2	New symbol per batch member; pointer placed in the `FuncTable`	Tier-1 callers land here after the entry-table swap; the thunk marshals tier-1 arguments into typed parameters and invokes the tier-2 body.
`t1_thunk`	Tier-2 → Tier-1	In-place rewrite of the non-batch stub function in the mini-module	Tier-2 batch bodies call non-batch helpers under the system convention; the rewritten body marshals typed arguments into a tier-1 args buffer, loads the helper pointer from the `FuncTable`, and dispatches under the tier-1 convention.
`entry_thunk`	LLVM-compiled → Tier-1	Per IR-JIT function; pointer stored on the function instance	Indirect calls from any LLVM-compiled body (tier-2, OSR, or whole-module LLVM) read this pointer through the proxy table and dispatch under the system convention; the thunk marshals into the tier-1 ABI internally.

Table 3.2: The three ABI bridge thunks. Each handles one direction of cross-tier dispatch; together they isolate tier-1 lowering from any awareness of the LLVM ABI and isolate the LLVM frontend from any awareness of the tier-1 ABI.

Each thunk is installed differently because each is reached differently by its callers. The LLVM frontend lowers every wasm function to a frontend function with a predictable name, and call instructions in batch bodies refer to that name directly.

doi:10.6342/NTU202601059



The `t1_thunk` is installed by overwriting the body of the existing stub function for a non-batch callee. Because the function name stays the same, the call instructions the frontend already emitted in batch bodies automatically dispatch through the new body. No call sites need rewriting.

The `fwd_thunk` is a fresh symbol. Tier-1 callers reach it through the `FuncTable` slot, not by name, so the symbol's name does not matter. A fresh symbol also lets the LLVM optimizer inline the tier-2 body into the thunk.

The `entry_thunk` is a fresh LLVM-convention wrapper around each tier-1 function. When LLVM-compiled code issues an indirect call to a tier-1 target, the call site reaches the entry_thunk under LLVM's convention, and the thunk dispatches to the real tier-1 function. Without it, an indirect call to a tier-1 target would fall back to a slow recovery path through the executor.

3.3.2 The tier-2 worker thread

A single background thread services every tier-2 compile request issued by the runtime. The thread is created lazily on the first notification and runs until the runtime shuts down. The choice of a single worker rather than a thread pool reflects two observations: tier-2 compile requests are infrequent relative to user-code execution, and serializing compiles avoids contention on the LLVM optimizer's internal data structures.

The worker maintains two request queues. A regular queue holds tier-2 batch requests issued by the function-entry notification helper; an OSR-priority queue holds OSR continuation-entry requests issued by the back-edge notification helper. When both queues are non-empty, the worker drains the OSR queue first. This priority ordering is justified

doi:10.6342/NTU202601059

by the structural difference between regular tier-2 batches and OSR requests. A regular tier-2 batch redirects future calls to the affected functions, but an OSR request migrates a currently-executing tier-1 frame. On a one-shot-caller workload (the common shape in serverless request handlers), the in-flight invocation is the only opportunity for OSR to deliver value. Delaying the OSR compile behind a regular batch can cause the in-flight loop to complete before the OSR entry is published.

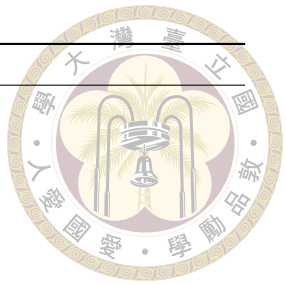
Two dedup sets prevent redundant work. One set tracks function indices for which a regular tier-2 batch has been enqueued. The other tracks (function, loop) pairs for which OSR continuation entries have been requested. The regular and OSR dedup sets are deliberately separate, because a function can be entry-promoted to tier-2 and have OSR continuation entries published for individual loops simultaneously.

3.3.3 Mini-module synthesis

The central design choice of tier-2 is to reuse the existing LLVM frontend without modification. Reimplementing wasm-to-LLVM lowering for tier-2 would duplicate a mature compiler. Instead, the architecture synthesizes a fresh wasm module that contains only the hot batch functions in their original form. Non-batch function bodies are replaced by stubs that allow the frontend's call lowering to resolve correctly. Algorithm 1 summarizes the synthesis.

The original module's section structure is preserved verbatim, except that non-batch defined functions receive trivial bodies. Each stub emits type-matched default constants for the function's declared return types.

doi:10.6342/NTU202601059



Algorithm 1 Mini-module synthesis for a tier-2 batch.

```
1:  procedure SynthesizeMiniModule(Src, BatchSet)
2:      Mini ← deep copy of Src
3:      for each defined function f in Mini do
4:          if f ∉ BatchSet then
5:              replace body of f with type-matched default returns
6:          end if
7:      end for
8:      mark Mini as validated
9:      return Mini
10: end procedure
```

3.3.4 Compile-ordering invariant

The tier-2 pipeline applies optimization in a specific order. The frontend compiles the mini-module unoptimized. The post-processing pass then installs the ABI bridge thunks and prunes unreferenced stubs. Only then does an aggressive optimization pass run. Reversing this order produces wrong-code bugs.

Before the bridges are installed, non-batch stubs appear to the optimizer as pure functions returning a constant. An aggressive pass inlines the constant into every cross-tier call site and eliminates everything that depended on it. By the time post-processing tries to install the real bridge, the call site no longer exists.

Compiling unoptimized first preserves every call site for the post-processing pass to rewrite. The subsequent optimization pass then sees real cross-tier dispatch bodies it cannot eliminate.

3.3.5 ABI bridging in the post-processing pass

The three thunks defined in Section 3.3.1 are installed at distinct points in the pipeline.

The `fwd_thunk` is added as a new symbol in the post-processing pass, after the un-

doi:10.6342/NTU202601059

optimized frontend compile. Each batch member receives one such thunk.

The `t1_thunk` is also installed in the post-processing pass, but by rewriting an existing stub body in place rather than by adding a new symbol.

The `entry_thunk` is built earlier, at module instantiation, before any user code runs. It is available the moment any LLVM-compiled body issues an indirect call.

3.3.6 Batch composition

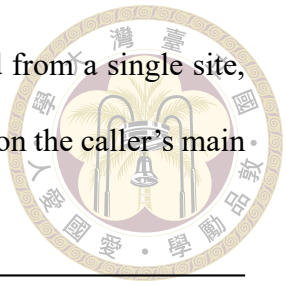
A tier-up event identifies a single hot function. Compiling that function in isolation hands the optimizer a body whose call sites all dispatch through `t1_thunks`. This blocks inlining of small helpers that account for substantial run-time on many kernels. The proposed batch composition algorithm therefore promotes the hot function together with a bounded set of its direct callees, all in one mini-module, so the optimizer can inline freely across the batch. Algorithm 2 summarizes the algorithm. Two design choices are worth noting.

First, the function that crosses the threshold is typically a small leaf called from a larger hot function. Compiling the leaf alone misses the surrounding context that contains most of the run-time. The algorithm therefore walks up the static call graph from the leaf, bounded to one hop, and selects the hottest direct caller as the batch root. From the root, it performs a bounded breadth-first traversal of direct callees.

Second, callee inclusion is gated by two tests. The dynamic test admits any callee whose dynamic call counter is non-zero; its purpose is to filter out dead code—functions that the static call graph identifies as reachable from the caller but that the workload never actually invokes. The static test admits any callee the caller's body references at least

doi:10.6342/NTU202601059

twice; its purpose is to filter out one-off references—callees invoked from a single site, which are typically rare-path or cleanup helpers rather than functions on the caller's main computation path. A callee enters the batch if either test passes.



Algorithm 2 Batch composition for a tier-up event at HotFuncIdx.

procedure ComposeBatch($HotFuncIdx$)
  $Root \leftarrow$ WalkUp($HotFuncIdx$) ▷ hottest direct caller, bounded depth
  $Batch \leftarrow \{Root,\ HotFuncIdx\}$
  $Frontier \leftarrow \{Root,\ HotFuncIdx\}$
  while $|Batch| < MaxSize$ **and** $Frontier \neq \emptyset$ **do**
    $C \leftarrow$ next direct callee from $Frontier$ at depth $\leq Limit$
    if Hot(C) **then**
      $Batch \leftarrow Batch \cup \{C\}$
    end if
  end while
  return $Batch$
end procedure

function Hot(C)
  return dynamic call count of $C > 0$ **or** static frequency to $C \geq StaticFreqThreshold$
end function

3.3.7 Atomic publication

After the optimization pass completes and the JIT compiler produces native pointers, the post-processor walks the LLVM module a final time to delete any stub function with no remaining users. It then atomically publishes the `fwd_thunk` pointer for each batch member into the `FuncTable` with release semantics.

The publication is a single atomic store. On the supported host architecture, the tier-1 dispatch load on the same slot has acquire ordering at the ISA level, so no explicit fence is required on the user thread. From the user thread's perspective, every direct dispatch through the slot either observes the tier-1 native pointer or the `fwd_thunk`, never an intermediate state.

doi:10.6342/NTU202601059

3.4 On-Stack Replacement

Function-entry promotion through the `FuncTable` swap accelerates future calls to a hot function but does not affect the currently-executing tier-1 frame. Some workloads call a function once and spend the bulk of execution inside a single long-running loop. This is the shape of cryptographic kernels, parsing loops, and many serverless request handlers. On these workloads, the in-flight invocation cannot benefit from tier-2, because the tier-1 frame never returns to its prologue.

The proposed on-stack replacement (OSR) mechanism closes this gap. It detects hot back-edges in tier-1 code and migrates the executing frame's wasm-level state into a tier-2 continuation entry mid-loop.

3.4.1 The one-shot-caller problem

The shape of the workloads that motivate OSR is illustrated by a class of cryptographic kernels in which a top-level function is invoked exactly once and spends nearly all of its run-time inside a tight numerical loop. Function-entry promotion of the top-level function delivers approximately zero speedup against the tier-1 baseline on such kernels, because the function never re-enters its prologue: the in-flight tier-1 frame runs the entire loop and returns. Whole-module LLVM JIT, in contrast, compiles the hot loop body with LLVM-grade optimization from the outset and runs it at full speed. OSR is the mechanism that lets a tiered architecture close this gap without paying the full LLVM compile cost up front.

doi:10.6342/NTU202601059

3.4.2 Three-state back-edge instrumentation

Every outermost loop in tier-1 code emits a back-edge instrumentation diamond that polls the loop's slot in the `OsrEntryTable` on every iteration. The slot encodes a three-state machine (Listing 3.1). In the counting state, the back-edge runs the per-loop counter. On saturation, the counter fires the OSR notification. The notification saturates the counter and writes a sentinel value that transitions the slot to the waiting state. In the waiting state, the back-edge falls through with no further work. This is the steady state on the post-saturation hot path, and reduces to three operations per iteration (load, test, branch). When the worker thread publishes the OSR continuation entry, the slot transitions to the ready state and holds the entry's native pointer. The next iteration of the loop snapshots the wasm locals into a per-thread buffer, tail-calls the entry, and returns its result.

```
slot = LOAD OsrEntryTable[loopId]
if slot != WAITING:                    // sentinel value 0
  if slot == COUNTING:                 // sentinel value 1
    counter = LOAD BackEdgeCounters[loopId]
    if counter < threshold:            // unsigned comparison
      counter = counter + 1
      STORE BackEdgeCounters[loopId] = counter
      if counter == threshold:
        notify_osr(loopId)
  else:                                // READY: slot is a function
      pointer
    snapshot wasm locals into a per-thread buffer
    return slot(env, locals)           // tail-call OSR entry, return its
        result
fall through to the back-edge
```

Listing 3.1: Back-edge instrumentation diamond (pseudocode).

The encoding compresses six operations per iteration (in an earlier two-array design

doi:10.6342/NTU202601059

that polled the counter array and an entry-pointer array independently) into three in the steady state. Because the back-edge runs on every loop iteration, this compression matters for the cumulative overhead OSR adds to the loop's hot path.

3.4.3 Continuation entry synthesis

When the OSR notification fires, the worker compiles a continuation entry that begins execution at the loop header of the targeted loop, with the current wasm locals as input parameters. The synthesis reuses the mini-module mechanism of Section 3.3.3 with three modifications.

The new function's parameters are the original function's parameters concatenated with its declared locals, flattened to a single parameter list. The return type is the original function's return type.

The new function's body consists of the structured-control opening sequence of the original loop's enclosing blocks, followed by the original instruction stream from the loop header through the function end. This shape ensures that branch-depth indices in the loop body resolve to the same labels they did in tier-1. Loops that cannot be safely re-entered mid-execution—those with non-empty block types, or those nested inside another control construct that requires entry from its prologue—are rejected at synthesis time and fall back to the tier-1 path.

Finally, the new function is appended to the module as a fresh function slot rather than overwriting the original. Intra-function calls within the OSR body therefore resolve to the original function index and dispatch back through tier-1 via the `t1_thunk`.

doi:10.6342/NTU202601059

3.4.4 OSR batch composition

OSR uses the same batch composition algorithm as regular tier-2 (Algorithm 2) with one parameter change: already-promoted callees are re-included in the OSR batch rather than left as `t1_thunk` dispatches.

The asymmetry reflects the OSR mini-module's role. A regular tier-2 batch shares its inlining work across all future calls to the batch members. An OSR batch, in contrast, exists to make this one loop fast in a single module. Dispatching helpers through `t1_thunks` on every iteration would block the optimizer from inlining short helpers into the loop body.

The worker-priority ordering described in Section 3.3.2 ensures that OSR compiles drain ahead of regular tier-2 batches. The reason is timing: the OSR transition window closes when the in-flight loop exits, and a continuation that lands after the loop has returned is a no-op.

doi:10.6342/NTU202601059

Chapter 4 Evaluation

This chapter evaluates the tiered compilation architecture. Section 4.1 introduces the benchmark suite, describes the strengthening procedure, defines the measurement protocol, and lists the hardware and parameters. Section 4.2 groups the kernels by execution shape; each tier-2 mechanism then maps to the group it targets. Section 4.3 establishes the whole-module LLVM JIT baseline as the upper bound on steady-state throughput. Sections 4.4–4.6 measure the tiered architecture in three configurations, one per design step. Section 4.7 reports compile time and end-to-end wall-clock time.

4.1 Methodology

4.1.1 The sightglass benchmark suite

Sightglass [4] is the WebAssembly benchmark suite maintained by the Bytecode Alliance. The suite ships a collection of kernels drawn from cryptographic primitives, parsing and serialization workloads, image and string processing, hashing, and small synthetic compute loops. Each kernel is compiled to WebAssembly through the standard toolchain and packaged with an input file and a golden output. The kernels are diverse enough to expose backend code-quality differences, and small enough that each kernel completes in

doi:10.6342/NTU202601059

seconds rather than minutes. This thesis uses Sightglass as an internal harness for comparing successive configurations of one runtime. It is not used as a cross-runtime benchmark. No claim of universal representativeness is made on its behalf.

The kernels are designed for cheap regression-testing. Most upstream kernels finish in well under a second on a modern host. The brevity is appropriate for correctness regression. The brevity is a measurement hazard for tier-2 evaluation. Tier-2 works by executing under the tier-1 baseline while a worker thread compiles the LLVM-grade replacement, then switching to the optimized code once it is ready. If the entire kernel completes before the switch, the measurement reflects only tier-1 work and reveals nothing about tier-2 code quality. Even when the switch fires, a residual tier-1 prefix of comparable size to the post-switch tail confounds the steady-state signal.

4.1.2 The sightglass-strong suite

To give tier-2 a measurable runway, this thesis strengthens every short kernel so that its tier-1-only run-time lands in a 5- to 10-second band, with a target near 8 seconds. The strengthened suite is referred to as sightglass-strong. A few seconds of post-switch execution is enough to amortize the worker's compile cost and to distinguish real steady-state speedup from measurement noise.

Three strengthening patterns cover most kernels. The first pattern increases an internal iteration constant defined at the top of the kernel source, used for shootout-style kernels whose inner loop is bounded by a single macro. The second pattern bumps the literal in the kernel's outer benchmark loop, used for kernels whose driver already loops over the work region. The third pattern wraps the benchmark region in a fresh outer loop,

doi:10.6342/NTU202601059

used for Rust and C kernels that originally measured a single execution. A small number of kernels require additional work: out-of-bounds fixes uncovered by the larger iteration count, optimization barriers to prevent the compiler from hoisting the kernel out of the surrounding loop, and replacement of nondeterministic output (such as wall-clock prints) so that the golden output remains stable. The per-kernel strengthening table is reproduced in Appendix A.

The evaluation also includes eight large real-world workloads that ship with upstream sightglass but sit outside its default suite: `spidermonkey-json`, `spidermonkey-markdown`, `spidermonkey-regex` (the SpiderMonkey JavaScript engine compiled to WebAssembly, running three different input scripts), `tinygo-json`, `tinygo-regex` (TinyGo-compiled Go programs), `image-classification` and `tract-onnx-image-classification` (ONNX-runtime image classifiers), and `sqlite3` (the SQLite database engine). These workloads expose the tiered architecture to module sizes and compile-cost profiles that the synthetic kernels do not reach. The full strengthened suite contains 41 measured kernels.

Goldens for sightglass-strong were regenerated by running the strengthened kernels under an independent runtime—wasmtime 43.0.1—rather than under WasmEdge itself. The independent oracle means that any regression in WasmEdge's parser, baseline JIT, or tier-2 pipeline surfaces as a golden mismatch. Without an independent oracle, the runtime output and the reference output would corrupt together silently.

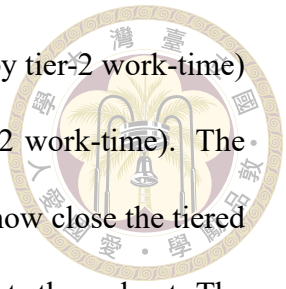
doi:10.6342/NTU202601059

4.1.3 Measurement protocol

Each measurement reports a three-run median to reduce single-run variance. All measurements are taken on the same host with the runtime built in release-optimized configuration and the tier-1 backend at its standard optimization level. Three metrics are reported. WorkTime (`WT`) is the harness-measured execution time of the benchmark region. Compile time (`Comp`) is the time spent compiling wasm functions to native code. The tier-1 path pays this cost before the first wasm instruction runs; the tier-2 path schedules it on the worker thread. Time-to-value (`TtV`) is the end-to-end wall-clock time from process start to benchmark completion (`Comp + WT` to a close approximation).

These three metrics map to the vocabulary used in the serverless and edge literature [15]. Cold-start latency is the time from request arrival at the runtime to the start of execution of the first machine-code instruction of the requested wasm function, on the assumption that no ready-to-run compiled instance of the module is already resident on the host. Under this definition, the full cold-start latency includes file I/O, parse, validate, compile, and instantiate. Compile dominates this sum on the kernels in the evaluation suite. The non-compile portion—load, parse, validate, and instantiate—is mode-independent: it runs the same code path under whole-module LLVM JIT and under the tiered IR-JIT path, contributes equally to both, and cancels out of every cross-mode ratio. This thesis therefore uses `Comp` as the operational measure of cold-start latency throughout the evaluation. The end-to-end wall-clock metric `TtV` is the composite of cold-start latency and steady-state execution; this thesis refers to it as "end-to-end wall-clock time" or `TtV`, never as "latency" alone, to avoid confusion with the cold-start sense. All ratios in this chapter follow the convention that values greater than one indicate a tier-2 win.

doi:10.6342/NTU202601059



The two ratios cited most often are `t1/t2` (tier-1 work-time divided by tier-2 work-time) and `LLVM/t2` (whole-module LLVM JIT work-time divided by tier-2 work-time). The configuration of interest is the second. The ratio `LLVM/t2` measures how close the tiered architecture comes to delivering whole-module LLVM JIT's steady-state throughput. The same convention applies to `Comp` and `TtV`: a ratio above one means tier-2 is faster. Per-kernel raw measurements for every configuration are reproduced in Appendix B.

4.1.4 Experiment setup

All measurements were taken on a single workstation-class host with no other interactive workload running. The host has a 13th-generation Intel Core i9-13900K processor (24 physical cores spanning performance and efficiency partitions, 32 hardware threads, 5.8 GHz turbo) and 128 GiB of DDR5 memory, running Ubuntu 22.04 LTS on Linux kernel 6.8. Each sightglass-strong run executes single-threaded. The worker thread that runs tier-2 compilation is the only other thread of interest. The worker is co-located with the user thread on the same host, but never contends for the same core during the benchmark region.

WasmEdge is built in release-optimized configuration. The tier-1 backend runs at its standard optimization level, which is the level the rest of the IR JIT pipeline uses. The whole-module LLVM JIT baseline runs through the unmodified WasmEdge LLVM frontend at its default optimization level, which is the level used for the production AOT path.

Three configurations are measured. Each configuration shares the same tier-1 baseline; the configurations differ only in which tier-2 mechanisms are enabled. Table 4.1

doi:10.6342/NTU202601059

summarizes the parameters.

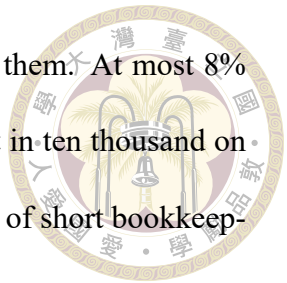
Configuration	Tier-2 promotion	Function-entry threshold	OSR threshold
Tier-1 only	disabled	—	—
Step 1 (hot callees)	enabled, leaf + direct callees	10 calls	legacy per-loop counter
Step 2 (walk-up + BFS)	enabled, walk-up + BFS	10 calls	OSR disabled
Step 3 (+ OSR)	enabled, walk-up + BFS	10 calls	5000 iterations
LLVM JIT baseline	not applicable	—	—

Table 4.1: Configurations used in this evaluation. The function-entry threshold is the number of calls a tier-1 function counts before triggering tier-2 promotion. The OSR threshold is the number of back-edge iterations a tier-1 loop counts before triggering an OSR continuation compile.

The two thresholds are kept fixed across kernels. Step 1 corresponds to the earliest tier-2 design measured on the `promote-hot-callees` branch (commit `f07da860`); Steps 2 and 3 correspond to the current head on the `osr` branch (commit `bdd80e91`). The tier-1 baseline numbers used in every step come from the current head, so the step-to-step deltas reflect changes to the tier-2 pipeline rather than incidental changes to tier-1.

The two threshold values are calibrated to the call-count and iteration-count distributions of the strengthened suite. The strengthening procedure tunes each kernel to a 5–10 second tier-1 work-region, which forces every kernel's hot functions to execute many thousands to many millions of times during measurement. A function-entry threshold of 10 therefore saturates within milliseconds of kernel start on every kernel, while remaining well above the call counts of one-time initialization, validation, and cleanup helpers that should not be queued. The OSR threshold sits an order of magnitude below the shortest one-shot-caller loop in the suite. The relevant iteration counts range from 60,000 (`shootout-ctype`, the shortest) to two billion (`shootout-random`), passing through 1.2 million (`shootout-base64`), 35 million (`shootout-matrix`), and 50 million

doi:10.6342/NTU202601059



(`shootout-ratelimit`). A threshold of 5000 fires on every one of them. At most 8% of the loop runs in tier-1 on `shootout-ctype`, and less than one part in ten thousand on `shootout-random`. The threshold also sits above the iteration counts of short bookkeeping loops, so the worker queue does not fill with continuations the user never re-enters. Both choices are robust over roughly a decade in either direction; the per-kernel statistics are several orders of magnitude away from the threshold on both sides.

4.1.5 Excluded kernels

Three kernels in the strong suite are excluded from the geometric means reported in this chapter. The kernel `noop` is excluded because its sub-microsecond run-time makes any ratio with seconds-scale kernels meaningless. The kernel `shootout-ackermann` is bimodal on the test host: depending on dynamic factors that have not been fully isolated, it lands in one of two well-separated time bands per run. The bimodality dominates per-kernel ratios involving this kernel. The kernel `splay` from upstream sightglass is not measured at all, because it uses WebAssembly garbage-collection instructions that tier-1 does not yet support; the kernel fails at tier-1 translation. The remaining 39 kernels constitute the analysis set.

A second class of upstream sightglass kernels is excluded one level earlier, before the strong suite is even assembled: kernels whose compiled wasm uses SIMD (`v128`) instructions, which tier-1 does not lower. The excluded directories are `blake3-simd`, `hex-simd`, `intgemm-simd`, `meshoptimizer`, and `libsodium` (the AEGIS, AES-GCM, and salsa20 families). For `blake3`, a scalar variant (`blake3-scalar`) is included in its place; the other four upstream directories have no scalar variant, so excluding the directory excludes the kernel from this evaluation entirely. The limitation is discussed in Section 5.2.

doi:10.6342/NTU202601059

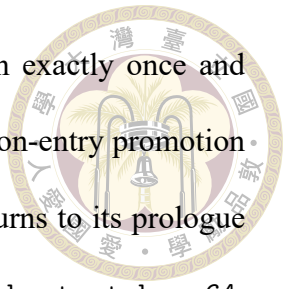
4.2 Workload Characterization

The 39 analysis kernels divide into five execution-shape categories. The categorization clarifies which tier-2 mechanism each kernel responds to. The categories are not strictly disjoint: two kernels (`shootout-ctype` and `blind-sig`) sit in two categories simultaneously, and the design's full effect on those kernels requires both mechanisms to engage.

Frequent-call workloads. The kernel invokes a hot function many times, with each call short relative to the total run-time. The function's entry counter saturates quickly, the entry-table swap captures every subsequent call, and the tier-1 prefix preceding the swap is negligible relative to the steady-state tail. Members: `blake3-scalar`, `bz2`, `gcc-loops`, `hashset`, `pulldown-cmark`, `quicksort`, `regex`, `shootout-gimli`, `shootout-heapsort`, `shootout-keccak`, `shootout-memmove`, `shootout-seqhash`, `shootout-sieve`, `shootout-switch`, `shootout-xblabla20`, `shootout-xchacha20`, `tinygo-regex`.

Hot-leaf workloads. The threshold-tripping function sits several layers below the actual hot work. A small leaf helper crosses its counter first, but the surrounding caller —which contains the bulk of the run-time—is what tier-2 needs to compile to deliver speedup. Promoting only the leaf yields a tier-2 body that calls back into tier-1 for every helper outside the batch, blocking inlining and pinning the call sites at bridge-thunk cost. Members: `blind-sig`, `rust-compression`, `rust-json`, `rust-protobuf`, `shootout-ctype`, `shootout-ed25519`.

doi:10.6342/NTU202601059



One-shot caller workloads. The kernel invokes its outer function exactly once and spends nearly all of its run-time inside one long-running loop. Function-entry promotion delivers no speedup on these kernels, because the function never returns to its prologue and never re-enters through the swapped slot. Members: `blind-sig`, `shootout-base64`, `shootout-ctype`, `shootout-matrix`, `shootout-minicsv`, `shootout-random`, `shootout-ratelimit`.

Resistant workloads. The kernel has either too little total run-time for the worker to amortize its compile cost, or structural properties that block optimizer wins. Members: `richards` (short total run-time), `rust-html-rewriter` (short total run-time), `shootout-fib2` (pure recursive call), `shootout-nestedloop` (innermost loop with no inline opportunity).

Compile-dominated workloads. The kernel's LLVM compile cost dwarfs its WebAssembly work-time. Steady-state throughput is largely irrelevant on these workloads; end-to-end wall-clock time is dominated by how long the runtime spends in compilation before the program produces useful output. Members: `image-classification`, `spidermonkey-json`, `spidermonkey-markdown`, `spidermonkey-regex`, `sqlite3`, `tinygo-json`, `tract-onnx-image-classification`. These are the workloads for which the cold-start architecture exists.

4.3 LLVM JIT Baseline

Whole-module LLVM JIT establishes the upper bound on steady-state throughput. Its work-time numbers are the divisor for every `LLVM/t2` ratio reported in this chapter. The

doi:10.6342/NTU202601059

baseline also defines the cost structure that the tiered architecture is designed to avoid.

LLVM JIT compiles every defined function before any wasm instruction executes. Its compile time therefore scales with module size and the complexity of individual function bodies. Three groups appear in the LLVM JIT compile-time distribution. Small modules with few or simple functions compile in tens to hundreds of milliseconds (`shootout-gimli` 17 ms, `shootout-heapsort` 82 ms, `shootout-keccak` 517 ms). Mid-sized modules compile in one to five seconds (`rust-html-rewriter` 4.7 s, `regex` 5.0 s, `rust-compression` 5.6 s). The eight large workloads added to the suite push the cost into the tens of seconds: `tinygo-regex` 7.8 s, `tinygo-json` 10.3 s, `sqlite3` 16.4 s, and the three SpiderMonkey kernels at approximately 90 s each. The ONNX-runtime classifier (`tract-onnx-image-classification`) compiles in 136 s.

End-to-end wall-clock (`TtV`) is the metric that matters for cold-start workloads. LLVM JIT's TtV includes its compile time in full. For the compile-dominated kernels, LLVM JIT's compile time is one to four orders of magnitude larger than its work time: `spidermonkey-json` compiles for 90.7 s and runs for 0.07 s; `tinygo-json` compiles for 10.3 s and runs for 0.05 s. The user waits a minute and a half for SpiderMonkey to start; the actual JavaScript execution finishes in under a tenth of a second. The tiered architecture's design goal is to reduce TtV on these workloads without losing more steady-state throughput than is necessary on the long-running kernels.

4.4 Step 1: Promote Hot Callees

The earliest tier-2 design promotes only the function whose counter saturated, together with its direct callees. The batch's root is the threshold-tripper itself; the breadth-

doi:10.6342/NTU202601059

first traversal extends one hop to its direct callees. There is no walk up the call graph.

Across the 39 analyzed kernels, the geometric mean of `t1/t2` is **0.97**× and the geometric mean of `LLVM/t2` is **0.76**×. Tier-2 in this configuration is on average slightly slower than tier-1, and recovers approximately 76% of LLVM JIT throughput.

The aggregate hides two distinct failure modes.

Hot-leaf failure. For kernels in the hot-leaf category, the threshold-tripper is a small helper. Compiling only the leaf yields a tier-2 body whose call sites all dispatch back into tier-1 through the ABI bridge thunks introduced in Section 3.3.1. The optimizer cannot inline across the thunk boundary, so the calls remain expensive. The kernels in this category show the largest gaps to LLVM JIT: `shootout-ctype` at 0.31×, `blind-sig` at 0.37×, `shootout-ed25519` at 0.38×. Each is below 40% of LLVM JIT throughput.

One-shot-caller failure. For kernels in the one-shot-caller category, the entry-table swap fires after the outer function has already started running. The in-flight invocation runs the entire loop in tier-1 and returns. The swap is observable only on subsequent calls, and those calls never happen. `shootout-ctype`, `blind-sig`, `shootout-random`, and `shootout-base64` all sit well below the geometric mean for this reason. Several kernels also regress relative to tier-1. The worker's compile cost is paid without a sufficient post-swap tail to amortize it: `hashset` runs at 0.73× `t1/t2`, `shootout-ed25519` at 0.66×, `shootout-ctype` at 0.54×.

The two improvements described in the following sections each address one of these failure modes.

doi:10.6342/NTU202601059

4.5 Step 2: Walk-Up and BFS Batch Composition

The improved batch composition walks up the static call graph from the threshold-tripper to find the hottest direct caller, then performs a bounded breadth-first traversal of that caller's direct callees. Callee inclusion is gated by the two-test admission rule described in Section 3.3.6. The walk-up rolls the surrounding caller into the batch, so the optimizer has full bodies to inline into.

Across the 39 kernels, the geometric mean of `t1/t2` rises to **1.03**× and the geometric mean of `LLVM/t2` rises to **0.80**×. Tier-2 now beats tier-1 by approximately 3% on average and recovers approximately 80% of LLVM JIT throughput.

The largest beneficiaries are kernels whose Step 1 batch was structurally too small for the optimizer to inline within (Figure 4.1). `hashset` rises from 0.62× to 0.84×, recovering from a Step 1 `t1/t2` regression to a clear tier-2 win. `shootout-ed25519` rises from 0.38× to 0.58× as the walk-up captures the cryptographic caller that wraps the modular-arithmetic leaf. The Rust kernels (`rust-json`, `rust-compression`, `rust-protobuf`) recover similarly: their static call graphs are deeper than the shootout kernels, so a single-hop BFS at Step 1 misses most of the hot work.

The size of the gain tracks the depth of the kernel's call graph. The shootout kernels have shallow graphs. On `hashset` and `shootout-ed25519`, the hot work sits one hop above the threshold-tripping leaf. A single walk-up hop captures the caller. A short breadth-first pass pulls in its helpers. On `shootout-ed25519`, the resulting batch is the cryptographic caller, the modular-arithmetic leaf, and five field-arithmetic helpers. The optimizer inlines the leaf into the caller. The gain is large.

doi:10.6342/NTU202601059

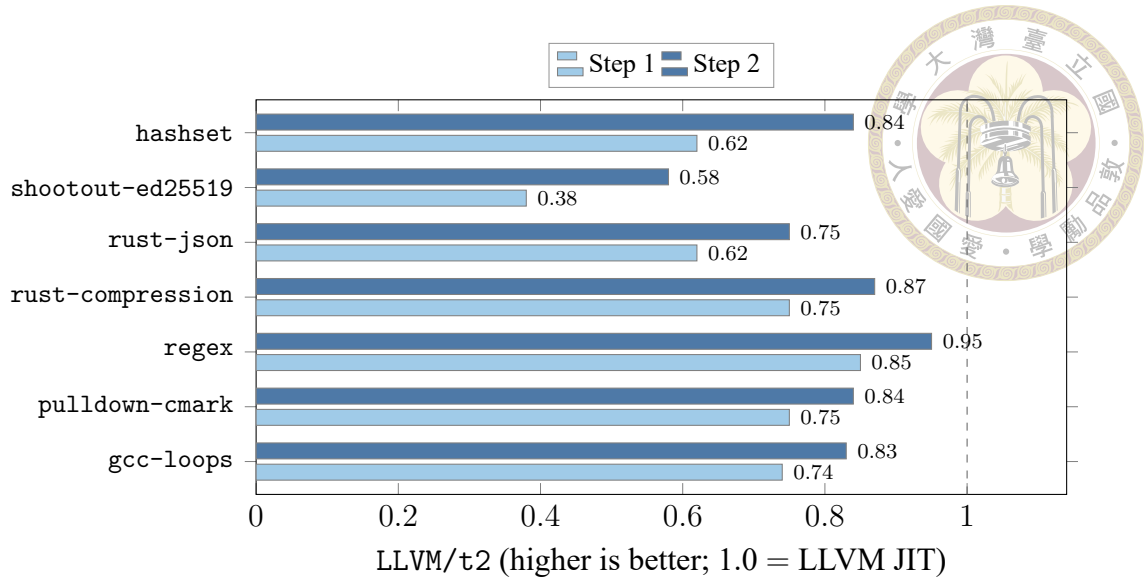
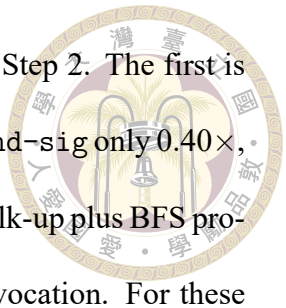


Figure 4.1: Steady-state throughput relative to whole-module LLVM JIT (`LLVM/t2`) for the seven kernels with the largest gain when batch composition switches from leaf-only to walk-up plus BFS (Step 1 → Step 2). The dashed line marks parity with LLVM JIT.

The Rust kernels have deep graphs. Rust expands generics and iterator chains into deep stacks of small functions. The hot work then sits many call levels above the leaf. Walk-up plus bounded breadth-first traversal engages across dozens of batches rather than one. It captures most of the hot path. Some hot callees still remain outside every batch, and they dispatch through bridge thunks on each call. This residue caps the recovery below parity. `rust-json` reaches 0.75× and `rust-compression` 0.87× at Step 2. The shallow kernels come close to LLVM JIT. The deep kernels do not. The parser kernels `regex` and `pulldown-cmark` follow the same deep-graph pattern. `gcc-loops` is a third shape. It is a collection of many separate loops. Tier-2 compiles each loop together with the small functions it calls.

The frequent-call kernels that were already in good shape at Step 1 show small changes. `shootout-keccak` moves from 0.97× to 0.98×, `shootout-gimli` from 0.99× to 0.99×, `blake3-scalar` from 0.96× to 0.99×. Function-entry promotion was already capturing the hot path on these kernels, and the larger batch the optimizer sees does not add additional inlining opportunities.

doi:10.6342/NTU202601059



Two kernel groups remain well below LLVM JIT throughput at Step 2. The first is the one-shot-caller group: `shootout-ctype` reaches only 0.37×, `blind-sig` only 0.40×, `shootout-random` only 0.63×. The function-entry promotion that walk-up plus BFS produces is correct and fires. It simply has no effect on the in-flight invocation. For these kernels, the in-flight invocation is the only invocation. Closing this group is the role of on-stack replacement. The second is the compile-dominated group: `spidermonkey-regex` reaches 0.55×, `tract-onnx-image-classification` 0.63×, `sqlite3` 0.68×. The steady-state gap on these workloads is structural: the hot paths use vector and floating-point patterns that the tier-1 backend lowers without LLVM-grade scheduling. Their end-to-end wall-clock time is the relevant metric instead, and it tells a different story (Section 4.7).

4.6 Step 3: On-Stack Replacement

The full configuration adds on-stack replacement (OSR) on top of walk-up and BFS. Tier-1 code now instruments each outermost-loop back-edge with the three-state diamond described in Section 3.4.2. When the back-edge counter for a loop saturates, the worker thread compiles a continuation entry whose entry point is the loop header. The currently-executing tier-1 frame migrates into the continuation entry on the next iteration.

Across the 39 kernels, the geometric mean of `t1/t2` rises to **1.11**× and the geometric mean of `LLVM/t2` rises to **0.86**×. Tier-2 with OSR beats tier-1 by approximately 11% on average and recovers approximately 86% of LLVM JIT throughput.

The one-shot-caller kernels show the largest swings (Figure 4.2). The seven kernels with the biggest `LLVM/t2` jump from Step 2 to Step 3 are all from this category. The two

doi:10.6342/NTU202601059

kernels at the top of the figure—`shootout-ctype` and `blind-sig`—swing more than half of the gap between Step 2 and LLVM JIT. Both kernels sit in both the hot-leaf and one-shot-caller categories. Walk-up at Step 2 produced a large optimizing-tier batch for each. The in-flight invocation never reached the swapped entry, so the batch was wasted. OSR delivers the in-flight migration that closes the remaining gap.

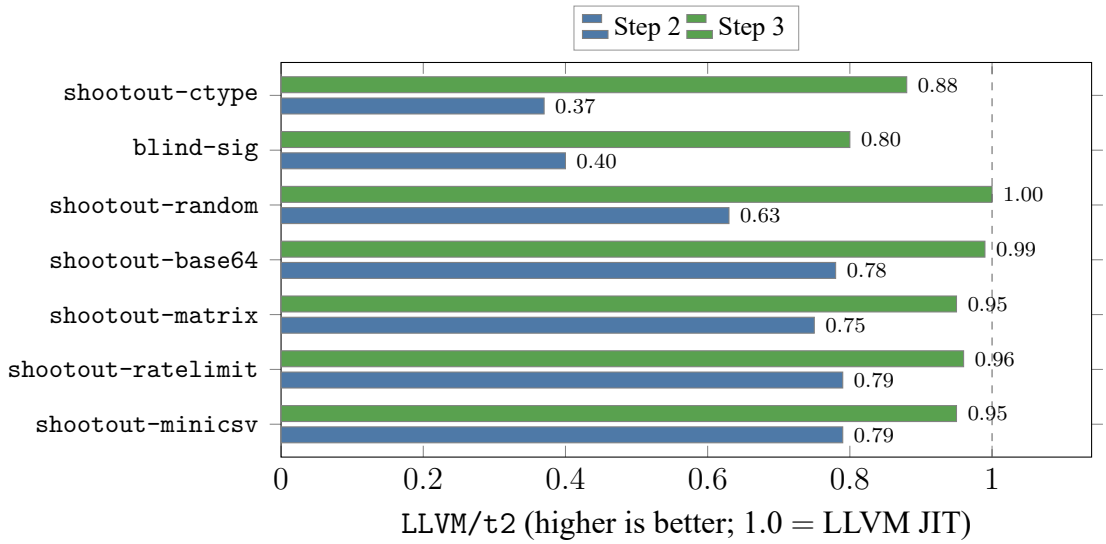
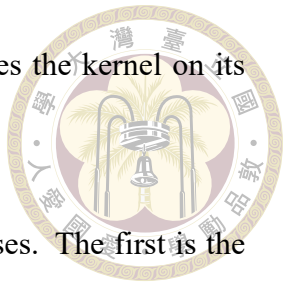


Figure 4.2: Steady-state throughput relative to whole-module LLVM JIT (`LLVM/t2`) for the seven kernels with the largest gain when OSR is enabled (Step 2 → Step 3). All seven are from the one-shot-caller category. The dashed line marks parity with LLVM JIT.

The mechanism behind each swing is identical. The outer function is called once. Almost all run-time is spent in one long-running loop. Without OSR, the function-entry slot swap fires after the loop has already started and is therefore observable only on a subsequent call that never happens. With OSR, the loop's back-edge counter saturates while the loop is still running, the worker compiles a continuation entry, and the tier-1 frame jumps into the continuation entry on the next iteration. The remainder of the loop runs in optimizing-tier code.

The five shootout kernels in this group share one structure. The runtime confirms it directly. Each compiles exactly one OSR continuation entry. The outer function is called once and spends its run-time in one loop. On `shootout-ratelimit`, function-

doi:10.6342/NTU202601059



entry promotion does almost nothing. The single continuation carries the kernel on its own.

The residual gap to LLVM JIT after OSR has two separate causes. The first is the length of the loop. OSR fires after a fixed back-edge count. A shorter loop therefore runs a larger fraction of its iterations in tier-1 before the migration. `shootout-ctype` has the shortest loop in the suite. Up to eight percent of it runs in tier-1, and that prefix alone holds it to 0.88×. `shootout-random` has the longest loop. Its tier-1 prefix is negligible, and it reaches parity at 1.00×. The second cause is the steady-state quality of the migrated loop body. When the prefix is negligible, the residual gap is how close the migrated body comes to whole-module LLVM. `shootout-base64` reaches 0.99× on a simple byte transform. `shootout-matrix` and `shootout-ratelimit` settle near 0.95× on heavier compute loops. The two causes are distinct. `shootout-base64` has a larger tier-1 prefix than `shootout-matrix`. It still reaches a higher final ratio. Prefix length alone cannot order the two.

`blind-sig` sits lowest in the table for a structural reason. It is not one loop. It is a large signature module. Many of its functions each carry a hot loop. The runtime installs dozens of continuation entries rather than one. The cross-tier boundaries between those migrated loops are the residue that holds it to 0.80×.

A small number of kernels regress when OSR is enabled. `rust-json` drops from 0.75× to 0.63×, `spidermonkey-json` from 0.80× to 0.70×, `pulldown-cmark` from 0.84× to 0.78×, `rust-html-rewriter` from 0.67× to 0.62×. These kernels share two properties: the inner loops are short relative to the rest of the program, and the hot functions are called frequently enough that function-entry promotion at Step 2 was already capturing

doi:10.6342/NTU202601059

the hot path. The added back-edge instrumentation costs three operations per iteration on the post-saturation steady state, which is small in absolute terms but visible when the loop body itself is short. The OSR continuation compile also competes with regular tier-2 batches for worker time. The OSR-priority drain (Section 3.3.2) ensures continuation entries land before the loop exits on the kernels for which OSR is the only mechanism that helps; the cost is a small delay for regular-tier-2 compiles on kernels for which OSR is unnecessary.

4.7 Compile Time and End-to-End Wall-Clock Time

The work-time analysis above is the steady-state metric. The compile metric is `Comp`; the end-to-end metric is `TtV`.

Across the 39 kernels, the geometric mean of `Comp T2/T1` in the full configuration is **3.05**×. The worker thread takes about three times as long as tier-1 alone before it produces a tier-2 body for the hot batch. The absolute gap is tens to hundreds of milliseconds on a typical kernel. The bulk of optimization runs off the critical path on the worker thread, so this is the cost the kernel pays for tier-2 compilation, not the cost the user thread observes. The ratio of LLVM JIT compile time to tier-2 compile time, `Comp LLVM/T2`, has a geometric mean of **11.1**×. Tier-2 finishes its compilation approximately eleven times faster than whole-module LLVM JIT on average. On the largest kernels the ratio is dominated by the absolute size of LLVM's compile work; on the smallest kernels the ratio is dominated by the fixed overhead of LLVM's pass pipeline.

End-to-end `TtV` integrates the trade-off. Across the 39 kernels, the geometric mean of `TtV LLVM/T2` is **1.73**×. The tiered architecture finishes the end-to-end workload approxi-

doi:10.6342/NTU202601059

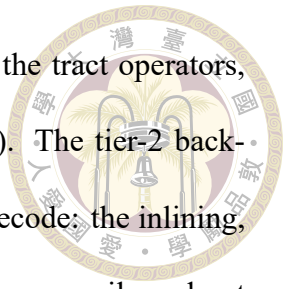
mately 73% faster than whole-module LLVM JIT on average, even though its steady-state throughput is 14% lower (0.86× `LLVM/t2`). The gain comes from the saved compile time.

The benefit is largest on the compile-dominated workloads. `tinygo-json` finishes in 473 ms under tier-2 versus 10,361 ms under LLVM JIT, a 21.9× wall-clock win. `tract-onnx-image-classification` finishes in 9.7 s under tier-2 versus 136 s under LLVM JIT, a 14.1× win. The three SpiderMonkey kernels each finish in approximately 9 s under tier-2 versus approximately 90 s under LLVM JIT, a 10× win. `sqlite3` finishes 8.5× faster, `image-classification` 8.8× faster. These are exactly the workloads the cold-start architecture targets: real applications and large modules where the user waits seconds to minutes for LLVM JIT to start, while the actual benchmark region takes milliseconds to seconds.

Why the large wins concentrate on interpreter-shaped wasm

The largest TtV wins are not scattered across the suite at random. They concentrate on a structurally distinct subset of the eight extension workloads—`spidermonkey-json`, `spidermonkey-markdown`, `spidermonkey-regex`, `tinygo-json`, `sqlite3`, `tract-onnx-image-classification`—and the structural reason matters for how the result generalizes. Each of these modules embeds a bytecode interpreter, a reflection-driven dispatch loop, or a graph of large tensor operators, compiled to wasm at the source toolchain's highest optimization level. SpiderMonkey runs JavaScript through its C++ bytecode dispatch loop, with the SpiderMonkey JIT tier disabled. SQLite runs SQL through its VDBE virtual machine—itself a computed-goto dispatch loop in C, pushed through Emscripten. TinyGo's `encoding/json.Unmarshal` dispatches every field decoder through `call_indirect`, resolved at run time from `reflect.Type` tables. The wasm-side hot

doi:10.6342/NTU202601059



path on each is opcode fetch followed by an indirect branch (or, on the tract operators, a single large function body whose tensor schedule is already fixed). The tier-2 backend cannot accelerate that hot path from already-optimized wasm bytecode: the inlining, vectorization, and dispatch choices are baked in at the source-to-wasm compile and not recoverable by re-optimizing the resulting wasm. The steady-state `LLVM/t2` ratio on these kernels is therefore flat by construction (Appendix A.3 documents the per-kernel structural reasons and the measured ratios).

The LLVM JIT compile cost on the same modules is not flat. It scales with module size, and these modules are large: 90 s for each of the three SpiderMonkey kernels, 136 s for the tract ONNX classifier, 16 s for SQLite, 10 s for the TinyGo JSON parser. The tiered architecture exists for this asymmetry. The user does not pay for steady-state speedup the backend cannot deliver. The architecture removes the dominant cold-start cost instead. That cost would otherwise gate any use of these modules in an edge or serverless deployment, where a cold-started JavaScript engine, SQL engine, or ONNX runtime is the actual unit of work.

The benefit is smallest on the long-running synthetic kernels, where LLVM JIT's compile cost is amortized over enough wasm execution that the wall-clock gap closes. `shootout-ctype` `TtV` is 0.93×, `shootout-sieve` 0.84×, `shootout-fib2` 0.90×: tier-2 is slightly slower end-to-end on these kernels because the steady-state regression is no longer offset by a meaningful cold-start saving. The bimodal split between the compile-dominated and work-dominated kernels is the central observation of this evaluation. The tiered architecture buys reduced cold-start latency at a modest cost in steady-state throughput. The trade-off pays off most clearly on the real-world workloads that motivate the architecture.

doi:10.6342/NTU202601059

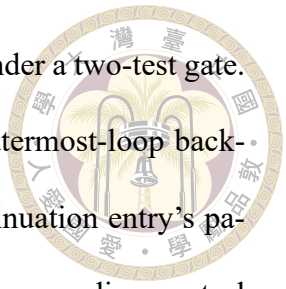
Chapter 5 Conclusion

5.1 Summary

This thesis presents a tiered compilation architecture for WasmEdge. The architecture decouples cold-start latency from peak code quality by routing the cold path through a lightweight Sea-of-Nodes baseline JIT, then selectively recompiling hot functions with the existing WasmEdge LLVM frontend on a background worker thread. A profile-driven promotion mechanism replaces baseline-tier entries with optimizing-tier entries atomically; an on-stack replacement mechanism migrates in-flight tier-1 frames into optimizing-tier continuation entries mid-loop.

Three design contributions support the architecture. The first is a tier-1 baseline JIT that integrates a third-party lightweight Sea-of-Nodes JIT library into WasmEdge as a compile-every-function pathway. The tier embeds per-function and per-loop profile counters, and exposes a uniform two-argument calling convention that unifies wasm-defined functions, host imports, and indirect-call targets. The second is a tier-2 selective recompilation pipeline that reuses the existing LLVM frontend through per-batch mini-module synthesis. The pipeline introduces three ABI bridge thunks that reconcile the tier-1 and LLVM calling conventions, a compile-ordering invariant that preserves cross-tier call sites for post-processing rewriting, and a batch composition algorithm that walks up the static

doi:10.6342/NTU202601059



call graph from the threshold-tripping leaf and admits direct callees under a two-test gate. The third is an on-stack replacement mechanism that instruments outermost-loop back-edges with a three-state diamond and lifts wasm locals into the continuation entry's parameter list. The mechanism bails out structurally on loops whose surrounding control flow cannot be safely re-entered mid-execution.

The architecture is implemented in WasmEdge and evaluated on the sightglass-strong benchmark suite, a strengthened variant of upstream sightglass. Each kernel's tier-1 runtime lands in a 5- to 10-second band, giving tier-2 a measurable post-swap runway. The evaluation includes the suite's larger real-world workloads—the SpiderMonkey JavaScript engine on three input scripts, two TinyGo-compiled Go programs, SQLite, and two ONNX-runtime image classifiers—which expose the architecture to module sizes the synthetic kernels do not reach. Across the 39 analyzed kernels, the full configuration delivers a geometric-mean speedup of 1.11× over the tier-1 baseline and recovers approximately 86% of whole-module LLVM JIT throughput, while reducing compile time by 11.1× on average. End-to-end wall-clock time—the metric that matters for cold-start workloads—improves by 1.73× on average over whole-module LLVM JIT. The largest wins concentrate on the real-world workloads, up to 21.9× on `tinygo-json` and 14.1× on the ONNX classifier. The step-by-step evaluation isolates the contribution of each mechanism: walk-up batch composition rescues the hot-leaf workloads (`shootout-ed25519` from 0.38× to 0.58×, `hashset` from 0.62× to 0.84×), and on-stack replacement closes the residual gap on one-shot-caller workloads (`shootout-ctype` from 0.37× to 0.88×, `blind-sig` from 0.40× to 0.80×, `shootout-random` from 0.63× to 1.00×).

doi:10.6342/NTU202601059

5.2 Limitations

The architecture has four structural limitations.

No SIMD lowering. Tier-1 lowers the full MVP instruction set, plus the reference-type, bulk-memory, and table extensions. It does not lower SIMD (`v128`); the baseline backend has no vector type. A function that uses `v128` therefore fails tier-1 compilation. Supporting it cheaply would mean scalarizing each `v128` operation into 64-bit operations, which would run far slower than LLVM's real vector instructions. The slowdown would measure the missing vector support, not the tiering mechanism. The evaluation therefore excludes the SIMD-compiled workloads: `libsodium`, `intgemm-simd`, `meshoptimizer`, `blake3-simd`, and `hex-simd`. Where a scalar variant exists, the evaluation uses it instead, as with `blake3-scalar`. Every measured workload then runs entirely on the compiled path, so the reported differences reflect compilation strategy, not instruction-set coverage.

No de-tiering. Once a function or loop is promoted to tier-2, the architecture provides no path back to tier-1. The decision is one-way. For long-running modules in which the workload shape changes over time, this design choice leaves no recovery option if a tier-2 specialization becomes a poor fit for a later phase of execution. The choice reflects a simplicity-versus-completeness trade-off appropriate to the cold-start use case the thesis targets, where the workload is typically short-lived.

x86-64 host support only. The tier-1 backend, the OSR continuation synthesis, and the ABI bridge thunks are all implemented and tested on x86-64. The underlying backend supports aarch64, and the thesis's modifications are largely architecture-neutral, but no

doi:10.6342/NTU202601059

aarch64 port has been validated. Edge deployments on ARM-based hosts are therefore not yet covered.

Worker-thread CPU contention on small hosts. The evaluation runs on a 24-core workstation in which the tier-2 worker thread executes on a physical core that the user thread never contends for. The compile-time costs reported in Chapter 4 are therefore off the critical path of the user thread. Edge nodes are frequently provisioned with one or two vCPUs, at which point the worker thread shares time with the user thread under the host scheduler. A long LLVM compile on a single-vCPU host will preempt the user thread for the duration of the compile, eroding the wall-clock benefit that the architecture targets. The mechanism is not broken in this case—tier-1 execution still proceeds, and the swap still fires correctly when the worker eventually completes—but the cold-start latency win is bounded by the host's ability to schedule the worker alongside, rather than in place of, useful execution. Mitigations are deployment-side rather than architecture-side: setting the worker thread to a lower scheduling priority, constraining the worker's CPU share through cgroups or affinity masks, or yielding cooperatively at coarse-grained LLVM-pass boundaries. None of these are implemented in the current prototype, and the appropriate policy depends on host characteristics that the runtime does not currently introspect.

5.3 Future Work

Four directions extend the architecture.

SIMD lowering and reference-type promotion scope. Adding tier-1 lowering for the WebAssembly SIMD (`v128`) instruction set is the primary missing piece. The lowering

doi:10.6342/NTU202601059

itself is mechanical—each SIMD opcode maps to one or a small group of IR backend SSE/AVX nodes—but the local-serialization buffer used by the OSR transition must also be widened to a type-native `v128` slot, and the interaction with the IR backend's common-subexpression elimination (which dictates the type-native-width design in the scalar case) must be re-validated for the wider type. Once tier-1 lowers these instructions, the scalar-only filter at the tier-2 promotion gate can be relaxed to admit `v128` and reference-type signatures, and the OSR transition buffer can be extended to marshal the additional types across the swap.

aarch64 port. Validating the tier-1 backend, the ABI bridge thunks, and the OSR continuation synthesis on aarch64 brings ARM-based edge hosts into scope. The bridge thunks must be re-derived for the AAPCS64 calling convention, and the OSR continuation entry's parameter layout must be re-validated against the aarch64 backend's register allocator.

Refcounted JIT cache for de-tiering. Adding a refcounted cache for tier-1 native code, indexed by function index and held alive while any frame may still observe the code, opens a path to de-tiering. A function or loop incorrectly promoted under one workload shape could be reverted to tier-1 once the workload shape changes, and re-promoted later with updated profile information.

Profile-guided batch policy. The current batch composition uses a fixed walk-up depth and a fixed BFS bound. Profile information beyond the per-function and per-loop counters —call-site frequencies, branch biases, value profiles—could drive a more selective batch policy that includes the surrounding context only when the profile indicates it will be inlined or specialized to advantage. The expected gain is on workloads where the current

doi:10.6342/NTU202601059

batch is too small (kernels still below 0.80× LLVM JIT after Step 3) and where the current batch is too large (kernels where the worker spends compile cycles on cold paths that the optimizer cannot prune).

doi:10.6342/NTU202601059

References

[1] C. F. Bolz, A. Cuni, M. Fijałkowski, and A. Rigo. Tracing the meta-level: PyPy's tracing JIT compiler. In Proceedings of the 4th Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems (ICOOOLPS), pages 18–25. ACM, 2009.

[2] Bytecode Alliance. Wasmtime baseline compilation RFC. https://github.com/bytecodealliance/rfcs/blob/main/accepted/wasmtime-baseline-compilation.md, 2022.

[3] Bytecode Alliance. Cranelift code generator. https://cranelift.dev/, 2024.

[4] Bytecode Alliance. Sightglass: A WebAssembly benchmark suite. https://github.com/bytecodealliance/sightglass, 2024.

[5] Bytecode Alliance. Wasmtime: A standalone WebAssembly runtime. https://docs.wasmtime.dev/, 2024.

[6] Y.-F. Chen, M.-H. Chen, C.-C. Huang, and C.-H. Tu. Accelerate RISC-V instruction set simulation by tiered JIT compilation. In Proceedings of the 16th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages (VMIL). ACM, 2024.

doi:10.6342/NTU202601059

[7] L. Clark. Making WebAssembly even faster: Firefox's new streaming and tiering compiler. Mozilla Hacks, https://hacks.mozilla.org/2018/01/making-webassembly-even-faster-firefoxs-new-streaming-and-tiering-compiler/, 2018.

[8] C. Click. Global code motion / global value numbering. ACM SIGPLAN Notices, 30(6):246–257, 1995.

[9] C. Click and M. Paleczny. A simple graph-based intermediate representation. In Papers from the 1995 ACM SIGPLAN Workshop on Intermediate Representations (IR'95), pages 35–49, 1995.

[10] Cloudflare, Inc. Cloudflare Workers documentation. https://developers.cloudflare.com/workers/, 2024.

[11] D. C. D'Elia and C. Demetrescu. On-stack replacement, distilled. ACM Transactions on Programming Languages and Systems, 40(2):7:1–7:50, 2018.

[12] Fastly, Inc. Fastly Compute. https://docs.fastly.com/products/compute, 2024.

[13] Fermyon Technologies, Inc. Spin: An open-source framework for WebAssembly microservices. https://developer.fermyon.com/spin, 2024.

[14] S. J. Fink and F. Qian. Design, implementation and evaluation of adaptive recompilation with on-stack replacement. In Proceedings of the International Symposium on Code Generation and Optimization (CGO), pages 241–252, 2003.

[15] P. Gackstatter, P. A. Frangoudis, and S. Dustdar. Pushing serverless to the edge with

doi:10.6342/NTU202601059

WebAssembly runtimes. In Proceedings of the 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid), pages 140–149, 2022.

[16] A. Haas. Dynamic tiering for WebAssembly in V8. V8 Blog, https://v8.dev/blog/wasm-dynamic-tiering, 2023.

[17] A. Haas and C. Backes. WebAssembly compilation pipeline in V8. V8 Developer Documentation, https://v8.dev/docs/wasm-compilation-pipeline, 2023.

[18] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien. Bringing the web up to speed with WebAssembly. In Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pages 185–200, 2017.

[19] U. Hölzle, C. Chambers, and D. Ungar. Debugging optimized code with dynamic deoptimization. In Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pages 32–43, 1992.

[20] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In Proceedings of the International Symposium on Code Generation and Optimization (CGO), pages 75–88, 2004.

[21] J. Michaud, Y. Suzuki, and S. Barati. Assembling WebAssembly. WebKit Blog, https://webkit.org/blog/11517/webassembly-tiering/, 2021.

[22] M. Paleczny, C. Vick, and C. Click. The java HotSpot server compiler. In Proceedings of the USENIX Java Virtual Machine Research and Technology Symposium (JVM), 2001.

doi:10.6342/NTU202601059

[23] D. Stogov. IR: Lightweight JIT compilation framework. https://github.com/dstogov/ir, 2024.

[24] S. Toub. Performance improvements in .NET 7. Microsoft DevBlogs, https://devblogs.microsoft.com/dotnet/performance_improvements_in_net_7/, 2022. Section: On-Stack Replacement.

[25] WasmEdge Project. WasmEdge: A high-performance WebAssembly runtime. Cloud Native Computing Foundation, https://wasmedge.org/, 2024.

[26] World Wide Web Consortium. WebAssembly core specification, version 2.0. W3C Recommendation, https://www.w3.org/TR/wasm-core-2/, 2022.

doi:10.6342/NTU202601059

Appendix A — Sightglass-Strong Kernel Details

This appendix documents the per-kernel strengthening procedure used to produce sightglass-strong from upstream sightglass, and lists the seven kernels that were retained at their upstream size because they resist tier-2 speedup.

A.1 Strengthening Categories

Three patterns cover most of the strengthening work. A small number of kernels need additional surgery; those are described in Section A.2. One kernel (`noop`) is left at upstream size as a harness-overhead reference.

A.1.1 Category A: Internal-constant bumps

The kernel exposes its iteration count as a single `#define` or named constant at the top of the source file. Strengthening increases the constant; no other changes are required.

doi:10.6342/NTU202601059

Kernel	Source file	Original	Strengthened
shootout-sieve	sieve.c	LENGTH 17,000	1,000,000
shootout-heapsort	heapsort.c	ITERATIONS 1,000	15,000
shootout-matrix	matrix.c	ITERATIONS 300,000	35,000,000
shootout-memmove	memmove.c	ITERATIONS 10	3,500
shootout-ratelimit	ratelimit.c	ITERATIONS 1,000,000	50,000,000
shootout-gimli	gimli.c	ITERATIONS 10,000	100,000,000
shootout-base64	base64.c	ITERATIONS 10,000	1,200,000
shootout-keccak	keccak.c	ITERATIONS 10,000	32,000,000
shootout-xblabla20	xblabla20.c	ITERATIONS 1,000	6,000,000
shootout-xchacha20	xchacha20.c	ITERATIONS 1,000	11,000,000
shootout-ctype	ctype.c	ITERATIONS 1,000	60,000
shootout-switch	switch.c	ITERATIONS 1,000	300,000
shootout-random	random.c	LENGTH 40,000,000	2,000,000,000
shootout-ed25519	ed25519.c	ITERATIONS 10,000	50,000
shootout-minicsv	minicsv.c	ITERATIONS 1,000,000	8,000,000

Table A.1: Category A: shootout-style kernels whose iteration count lives in a top-of-file `#define`.

A.1.2 Category B: Main-loop literal bumps

The kernel's `main` function contains an outer loop with a numeric literal; strengthening replaces the literal.

Kernel	Source file	Change
quicksort	quicksort.c	outer loop 100 → 40,000 (plus an out-of-bounds fix; see Section A.2)
richards	richards.c	outer loop 100 → 35 (an earlier 10,000 timed out at over 120 s; 500 still overshot at 115 s)
hashset	HashSet.cpp	outer loop 100 → 30,000 (plus a golden-determinism fix)
gcc-loops	gcc-loops.cpp	`Mi` $1 \ll 18 \rightarrow 660{,}000$

Table A.2: Category B: kernels whose driver loop already exists; strengthening edits the literal.

A.1.3 Category C: Outer-loop wraps

The kernel measures a single execution of the benchmark region. Strengthening wraps that region in a new outer loop. All wraps sit between the kernel's `bench_start` and

doi:10.6342/NTU202601059

bench_end markers; any one-time print statements that depend on final state are moved outside the loop so the golden output stays deterministic.

Kernel	Source file	Wrap iteration count
bz2	benchmark.c	400 (compress + decompress per iteration)
blake3-scalar	main.rs	100,000 (`blake3::hash` per iteration)
rust-compression	main.rs	10 (gzip and brotli codecs per iteration)
rust-json	main.rs	1,600 (parse and serialize per iteration)
pulldown-cmark	main.rs	2,500 (parse and render into a fresh output buffer per iteration)
regex	main.rs	200 (three `count_matches` calls per iteration)
blind-sig	main.rs	200 (with the first iteration outside the wrap to initialize the signature)
rust-html-rewriter	main.rs	800 (`rewrite_str` per iteration)
rust-protobuf	main.rs	2,500 (decode and encode per iteration)
tinygo-regex	main.go	7 (three regex pattern counts per iteration; TinyGo `opt=2 -gc=leaking`)

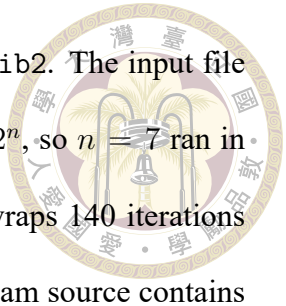
Table A.3: Category C: kernels that originally measured a single execution; strengthening wraps the bench region in a fresh outer loop.

A.2 Special Cases

Five kernels need work beyond a constant bump. Three of them ran afoul of compiler optimizations that hoisted the benchmark region out of the new outer loop. Two had latent out-of-bounds bugs that were masked at upstream iteration counts but trapped under the strengthened ones.

shootout-fib2. Pure recursive Fibonacci. The outer loop wraps fourteen iterations; without an optimization barrier on the input `n`, the loop-invariant code motion pass hoists the recursive call out of the loop entirely (the recursive function is pure and `n` is a local literal). A `BLACK_BOX(n)` call inside the loop body forces a real call per iteration.

doi:10.6342/NTU202601059



shootout-ackermann. Same hoisting failure mode as `shootout-fib2`. The input file is bumped from $n = 7$ to $n = 10$ (Ackermann's growth is roughly 2^n, so $n = 7$ ran in 5 ms while $n = 10$ runs in 60–80 ms per call), and the outer loop wraps 140 iterations with optimization barriers on both Ackermann arguments. The upstream source contains an unrelated bug—a buffer declared `char* buf[32]` (array of pointers) where the code expects `char buf[32]` (array of bytes)—left in place to avoid diverging further from upstream.

shootout-nestedloop. Six nested loops over an integer counter. The clang scalar-evolution pass collapses the entire nest into a closed-form polynomial when the body is simple enough. The body was rewritten from `x++` to `x = x*31 + f` and the counter declared `volatile int`; the rewrite alone is not enough, because the polynomial collapse keeps working on the new form. The `volatile` qualifier forces a real load and store on each iteration, which defeats the pass.

shootout-seqhash. The `HASH_ITERATIONS` constant is bumped from 1 to 3, but the kernel's stack-allocated array is sized for `LEVELS` entries while the body writes `LEVELS * HASH_ITERATIONS`. The latent overflow trapped at the strengthened size and was fixed by re-declaring the array with the larger product. The `ARENA_SIZE` constant was also halved (from 1,000,000 to 500,000) because scaling `HASH_ITERATIONS` alone was non-linear; halving the arena restores linear scaling.

quicksort. A `printf` reads past the end of the sort array when the loop variable exceeds `sortelements` (5,000). The 40,000-iteration bump causes the loop variable to exceed 5,000 routinely. Fixed by modular indexing: `sortlist[(run % sortelements) + 1]`.

doi:10.6342/NTU202601059

hashset. The kernel printed a per-run wall-clock timing string (`"That took X seconds."`) that made the golden output unstable across runs. Replaced with a fixed `"Done."` string; the surrounding wall-clock captures were dropped.

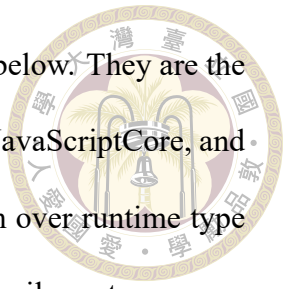
A.3 Tier-2-Resistant Kernels

Seven kernels in the suite are retained at their upstream iteration counts rather than strengthened: `spidermonkey-json`, `spidermonkey-markdown`, `spidermonkey-regex`, `tinygo-json`, `image-classification`, `sqlite3`, and `tract-onnx-image-classification`. Each was strengthened to land in the 5–10 second tier-1 band and measured against tier-2 (regular) and tier-2 with OSR. The results are reproduced in Table A.4. In every case, the strengthened tier-2/tier-1 work-time ratio came in within $\pm 0.05\times$ of $1.0\times$—often below $1.0\times$. The strengthened versions would inflate the suite's geometric means toward $1.0\times$ without telling a new story, so the kernels are kept at upstream size and counted as evidence that tier-2 cannot accelerate these workload shapes. The compile-time difference between LLVM JIT and the tiered architecture on these kernels is, however, real and substantial; the cold-start `TtV` analysis in Section 4.7 uses exactly these kernels as its showcase.

Strengthened kernel	T1 (μs)	T2 (μs)	T2+OSR (μs)	T1/T2	T1/(T2+OSR)
spidermonkey-json (100 iters)	6,861,438	6,841,687	7,679,136	1.00×	0.89×
spidermonkey-markdown (150 iters)	5,839,787	6,263,446	6,547,835	0.93×	0.89×
spidermonkey-regex (350 iters)	8,105,837	8,747,908	7,884,831	0.93×	1.03×
tinygo-json (120 iters)	7,895,398	7,844,385	7,826,273	1.01×	1.01×
image-classification (3,500 iters)	5,067,486	5,059,485	4,937,057	1.00×	1.03×
sqlite3 (10 iters)	8,208,141	7,962,978	8,672,875	1.03×	0.95×
tract-onnx-image-classification (30 iters)	7,720,191	7,974,772	8,284,838	0.97×	0.93×
tinygo-regex (7 iters)	8,468,501	6,185,582	6,231,380	**1.37**×	**1.36**×

Table A.4: The tier-2-resistant kernels: seven kernels whose strengthened versions produce flat tier-2 ratios, plus `tinygo-regex` (the only one of the eight extension kernels that does produce a real tier-2 win, retained at its strengthened size).

doi:10.6342/NTU202601059



The structural reasons each kernel resists tier-2 are documented below. They are the same patterns that limit multi-tier JITs in native runtimes such as V8, JavaScriptCore, and .NET: native FFI hops, wasm-compiled interpreters, indirect dispatch over runtime type information, and large-function workloads with high per-function compile cost.

image-classification. Host-bound. The bench region wraps a single `wasi_nn::compute` call; the host plugin performs the matrix work in native AVX. The wasm portion is microseconds next to milliseconds of host work, so tier-2 has nothing left to optimize.

spidermonkey-{json, markdown, regex}. Interpreter-shaped wasm. Each kernel is the SpiderMonkey C++ engine compiled to wasm with the SpiderMonkey JIT tier disabled. JavaScript therefore runs through SpiderMonkey's bytecode interpreter: a large `clang -O3`-compiled dispatch switch. The hot inner loop is opcode fetch plus dispatch, bottlenecked by branch prediction and instruction fetch rather than codegen quality. The tier-2 pipeline cannot recover the C++ semantics from already-O3-compiled wasm bytecode; inlining, vectorization, and loop choices are baked in at the C++-to-wasm step.

tinygo-json. Reflection-heavy wasm. The standard-library `encoding/json.Unmarshal` dispatches every field decoder through a `call_indirect` resolved at run time from `reflect.Type` tables. Neither tier-1 nor tier-2 can devirtualize across `call_indirect` from wasm bytecode alone; the call target depends on values that constant folding cannot reach.

sqlite3. Interpreter-shaped wasm at smaller scale than SpiderMonkey. SQLite's VDBE is itself a bytecode interpreter: a computed-goto dispatch loop in C, pushed through Emscripten `-O3` at compile time. The remaining wasm-side cycles spread thinly across many

doi:10.6342/NTU202601059

small functions (parser, btree, pager); few of them individually cross the tier-2 promotion threshold, so tier-2 promotes only a handful of functions and the steady-state delta is in the noise.

tract-onnx-image-classification. Compile-cost-dominated. The tract tensor operators produce large wasm functions, and the per-function LLVM compile cost is steep enough that enabling tier-2 inflates the compile-time column from 3.5 s to 8.3 s. The tier-2 swap therefore arrives late in the bench window and leaves little post-swap runway. The whole-module LLVM JIT row tells the same story from the other side: 138 s of upfront compile for 167 ms of execution.

tinygo-regex (retained). Direct-called scalar code. The regex engine compiles a deterministic finite automaton at build time, and the match loop is a tight scan over a byte slice with scalar comparisons and byte loads. Direct calls, regular loops, and scalar arithmetic are exactly the shape where the LLVM optimizer improves on the tier-1 backend, so this kernel is retained at the strengthened size and contributes a real tier-2 win to the main aggregates.

doi:10.6342/NTU202601059

Appendix B — Raw Measurement Tables

This appendix reproduces the per-kernel measurements that the geometric means and selected rows in Chapter 4 draw from. Every value is a three-run median. The kernel `noop` is included as a sanity row but is excluded from every geometric mean reported in the chapter. The bimodal kernel `shootout-ackermann` is included in each per-configuration table but is also excluded from the geometric means.

All ratios follow the convention that values greater than one indicate a tier-2 win: `T1/T2` = tier-1 work-time divided by tier-2 work-time, `LLVM/T2` = whole-module LLVM JIT work-time divided by tier-2 work-time, and the same direction is used for the compile-time and `TtV` ratios.

B.1 LLVM JIT Baseline

Whole-module LLVM JIT (`WASMEDGE_SIGHTGLASS_MODE=JIT`). These columns are the divisors for every `LLVM/T2` ratio in Sections B.3–B.5.

doi:10.6342/NTU202601059

Table B.1: LLVM JIT baseline: WorkTime, compile time, and end-to-end `TtV`.

Kernel	WT (μs)	Comp (ms)	TtV (ms)
blake3-scalar	5,200,415	687	5,882
blind-sig	4,156,440	3,095	7,351
bz2	7,060,749	1,102	8,168
gcc-loops	6,806,592	3,723	10,566
hashset	4,940,947	1,991	6,903
image-classification	2,784	2,599	2,667
pulldown-cmark	2,179,733	1,915	4,099
quicksort	6,805,066	215	7,024
regex	8,603,822	4,993	13,614
richards	732,598	75	810
rust-compression	6,999,568	5,556	12,565
rust-html-rewriter	879,841	4,711	5,595
rust-json	2,181,605	1,760	3,945
rust-protobuf	2,064,158	993	3,060
shootout-ackermann	3,482,395	303	3,787
shootout-base64	6,596,252	281	6,879
shootout-ctype	4,997,939	263	5,269
shootout-ed25519	4,852,592	2,517	7,371
shootout-fib2	5,665,627	205	5,869
shootout-gimli	8,067,482	17	8,085
shootout-heapsort	7,952,058	82	8,038

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	WT (μs)	Comp (ms)	TtV (ms)
shootout-keccak	6,859,866	517	7,378
shootout-matrix	6,771,200	274	7,046
shootout-memmove	2,645,052	284	2,926
shootout-minicsv	1,464,896	54	1,519
shootout-nestedloop	8,886,440	203	9,095
shootout-random	4,372,971	203	4,576
shootout-ratelimit	866,915	276	1,143
shootout-seqhash	5,527,015	311	5,846
shootout-sieve	7,173,303	198	7,372
shootout-switch	9,016,662	954	9,980
shootout-xblabla20	2,530,955	288	2,820
shootout-xchacha20	7,966,930	280	8,248
spidermonkey-json	65,747	90,669	90,880
spidermonkey-markdown	36,193	89,981	90,142
spidermonkey-regex	14,900	90,265	90,396
sqlite3	563,305	16,377	16,953
tinygo-json	47,576	10,304	10,361
tinygo-regex	5,337,782	7,791	13,144
tract-onnx-image-classification	169,237	135,938	136,445
noop	0	56	56

doi:10.6342/NTU202601059

B.2 Tier-1 Baseline

Tier-1 IR JIT (`WASMEDGE_SIGHTGLASS_MODE=IR_JIT`, tier-2 disabled). These columns are the divisors for every `T1/T2` ratio. The same tier-1 numbers are used in every step.

Table B.2: Tier-1 IR JIT baseline: WorkTime, compile time, and end-to-end `TtV`.

Kernel	WT (μs)	Comp (ms)	TtV (ms)
blake3-scalar	5,353,822	15	5,370
blind-sig	9,373,218	87	9,467
bz2	8,577,998	34	8,615
gcc-loops	9,151,021	156	9,317
hashset	5,804,478	762	6,586
image-classification	2,199	71	147
pulldown-cmark	3,043,596	60	3,108
quicksort	8,054,768	6	8,061
regex	10,114,573	123	10,261
richards	903,620	2	906
rust-compression	10,567,569	181	10,761
rust-html-rewriter	1,218,556	118	1,348
rust-json	3,458,560	41	3,504
rust-protobuf	2,837,580	22	2,864
shootout-ackermann	4,694,035	8	4,703
shootout-base64	8,433,085	8	8,442
shootout-ctype	8,788,804	8	8,799

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	WT (μs)	Comp (ms)	TtV (ms)
shootout-ed25519	8,451,759	147	8,588
shootout-fib2	7,886,532	6	7,893
shootout-gimli	8,088,746	0	8,089
shootout-heapsort	9,110,507	2	9,113
shootout-keccak	8,292,107	3	8,295
shootout-matrix	9,116,232	8	9,125
shootout-memmove	2,728,614	8	2,737
shootout-minicsv	2,095,122	1	2,096
shootout-nestedloop	5,439,761	6	5,446
shootout-random	6,927,647	6	6,934
shootout-ratelimit	1,085,281	8	1,094
shootout-seqhash	5,477,030	9	5,486
shootout-sieve	8,678,701	6	8,686
shootout-switch	8,492,810	36	8,531
shootout-xblabla20	3,070,611	8	3,079
shootout-xchacha20	8,398,324	9	8,408
spidermonkey-json	81,095	3,133	3,387
spidermonkey-markdown	54,853	3,195	3,409
spidermonkey-regex	26,367	3,132	3,312
sqlite3	828,592	340	1,189
tinygo-json	67,877	275	355

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	WT (μs)	Comp (ms)	TtV (ms)
tinygo-regex	8,360,584	216	8,588
tract-onnx-image-classification	270,125	3,565	4,251
noop	0	1	1

B.3 Step 1: Promote Hot Callees

Tier-2 arm from the `promote-hot-callees` branch (commit `f07da860`). Rows sorted by `LLVM/T2` descending, with `noop` at the bottom.

Table B.3: Step 1 (Promote Hot Callees) WorkTime and ratios. Geomean over 39 kernels (`noop` and `shootout-ackermann` excluded): `T1/T2` = 0.972×, `LLVM/T2` = 0.756×.

Kernel	T2 WT (μs)	T1/T2	LLVM/T2
shootout-nestedloop	5,440,386	1.00×	1.63×
shootout-ackermann	3,108,238	1.51×	1.12×
shootout-switch	8,595,380	0.99×	1.05×
shootout-seqhash	5,429,584	1.01×	1.02×
shootout-fib2	5,708,687	1.38×	0.99×
shootout-gimli	8,158,523	0.99×	0.99×
shootout-memmove	2,721,678	1.00×	0.97×
shootout-keccak	7,061,055	1.17×	0.97×
quicksort	7,019,701	1.15×	0.97×

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	T2 WT (μs)	T1/T2	LLVM/T2
blake3-scalar	5,406,670	0.99×	0.96×
shootout-xchacha20	8,419,702	1.00×	0.95×
image-classification	2,961	0.74×	0.94×
bz2	7,590,988	1.13×	0.93×
shootout-heapsort	9,094,983	1.00×	0.87×
regex	10,100,564	1.00×	0.85×
shootout-xblabla20	3,002,297	1.02×	0.84×
tinygo-regex	6,583,796	1.27×	0.81×
shootout-sieve	8,922,085	0.97×	0.80×
spidermonkey-json	82,917	0.98×	0.79×
richards	927,574	0.97×	0.79×
shootout-ratelimit	1,102,093	0.98×	0.79×
shootout-base64	8,466,227	1.00×	0.78×
rust-compression	9,292,646	1.14×	0.75×
pulldown-cmark	2,912,271	1.05×	0.75×
shootout-matrix	9,104,066	1.00×	0.74×
gcc-loops	9,193,173	1.00×	0.74×
shootout-minicsv	2,083,213	1.01×	0.70×
tinygo-json	68,337	0.99×	0.70×
sqlite3	836,743	0.99×	0.67×
rust-html-rewriter	1,359,336	0.90×	0.65×

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	T2 WT (μs)	T1/T2	LLVM/T2
rust-protobuf	3,231,314	0.88×	0.64×
spidermonkey-markdown	57,066	0.96×	0.63×
shootout-random	6,943,921	1.00×	0.63×
tract-onnx-image-classification	270,004	1.00×	0.63×
hashset	7,916,773	0.73×	0.62×
rust-json	3,545,115	0.98×	0.62×
spidermonkey-regex	27,119	0.97×	0.55×
shootout-ed25519	12,846,623	0.66×	0.38×
blind-sig	11,386,847	0.82×	0.37×
shootout-ctype	16,316,727	0.54×	0.31×
noop	0	—	—

Table B.4: Step 1 (Promote Hot Callees) compile time and `TtV`. Geomean over 39 kernels: `Comp T2/T1` = 1.009×, `Comp LLVM/T2` = 33.724×, `TtV T2/T1` = 1.001×, `TtV LLVM/T2` = 1.797×.

Kernel	Comp T2 (ms)	LLVM/T2 Comp	TtV T2 (ms)	LLVM/T2 TtV
blake3-scalar	16	42.10×	5,425	1.08×
blind-sig	88	35.05×	11,483	0.64×
bz2	35	31.06×	7,629	1.07×
gcc-loops	167	22.25×	9,372	1.13×
hashset	719	2.77×	8,649	0.80×

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	Comp T2 (ms)	LLVM/T2 Comp	TtV T2 (ms)	LLVM/T2 TtV
image-classification	75	34.77×	152	17.52×
pulldown-cmark	61	31.45×	2,978	1.38×
quicksort	6	34.90×	7,026	1.00×
regex	129	38.74×	10,260	1.33×
richards	2	34.39×	930	0.87×
rust-compression	186	29.79×	9,499	1.32×
rust-html-rewriter	126	37.47×	1,495	3.74×
rust-json	44	40.09×	3,594	1.10×
rust-protobuf	23	42.51×	3,258	0.94×
shootout-ackermann	9	34.76×	3,118	1.21×
shootout-base64	8	34.50×	8,475	0.81×
shootout-ctype	8	32.37×	16,330	0.32×
shootout-ed25519	122	20.60×	12,974	0.57×
shootout-fib2	6	33.72×	5,715	1.03×
shootout-gimli	0	44.01×	8,159	0.99×
shootout-heapsort	2	34.54×	9,098	0.88×
shootout-keccak	3	165.88×	7,065	1.04×
shootout-matrix	8	33.94×	9,113	0.77×
shootout-memmove	8	35.36×	2,734	1.07×
shootout-minicsv	1	50.47×	2,084	0.73×
shootout-nestedloop	7	31.21×	5,448	1.67×

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	Comp T2 (ms)	LLVM/T2 Comp	TtV T2 (ms)	LLVM/T2 TtV
shootout-random	7	29.43×	6,950	0.66×
shootout-ratelimit	8	33.32×	1,111	1.03×
shootout-seqhash	9	35.44×	5,439	1.07×
shootout-sieve	6	32.38×	8,930	0.83×
shootout-switch	36	26.20×	8,640	1.16×
shootout-xblabla20	8	35.35×	3,011	0.94×
shootout-xchacha20	9	32.14×	8,431	0.98×
spidermonkey-json	2,381	38.08×	2,645	34.36×
spidermonkey-markdown	2,397	37.54×	2,613	34.50×
spidermonkey-regex	2,396	37.68×	2,578	35.07×
sqlite3	363	45.11×	1,217	13.93×
tinygo-json	281	36.61×	361	28.70×
tinygo-regex	224	34.80×	6,832	1.92×
tract-onnx-image-classification	3,663	37.11×	4,371	31.22×
noop	1	—	1	—

B.4 Step 2: Walk-Up Plus BFS (no OSR)

Tier-2 arm from the `osr` branch (commit `bdd80e91`) with the OSR threshold set to zero. Rows sorted by `LLVM/T2` descending.

doi:10.6342/NTU202601059

Table B.5: Step 2 (Walk-Up + BFS) WorkTime and ratios. Geomean over 39 kernels: `T1/T2` = 1.033×, `LLVM/T2` = 0.804×.

Kernel	T2 WT (μs)	T1/T2	LLVM/T2
shootout-nestedloop	5,431,988	1.00×	1.64×
shootout-ackermann	2,774,818	1.69×	1.25×
image-classification	2,248	0.98×	1.24×
shootout-switch	8,614,484	0.99×	1.05×
shootout-seqhash	5,417,119	1.01×	1.02×
blake3-scalar	5,233,309	1.02×	0.99×
shootout-gimli	8,153,746	0.99×	0.99×
shootout-keccak	6,971,609	1.19×	0.98×
bz2	7,187,289	1.19×	0.98×
shootout-memmove	2,707,690	1.01×	0.98×
quicksort	6,996,109	1.15×	0.97×
regex	9,014,088	1.12×	0.95×
shootout-xchacha20	8,434,705	1.00×	0.94×
shootout-heapsort	9,061,037	1.01×	0.88×
tinygo-regex	6,086,568	1.37×	0.88×
rust-compression	8,044,600	1.31×	0.87×
shootout-fib2	6,515,013	1.21×	0.87×
pulldown-cmark	2,584,154	1.18×	0.84×
shootout-xblabla20	3,007,163	1.02×	0.84×
hashset	5,894,940	0.98×	0.84×

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	T2 WT (μs)	T1/T2	LLVM/T2
gcc-loops	8,201,099	1.12×	0.83×
richards	908,661	0.99×	0.81×
shootout-sieve	8,902,376	0.97×	0.81×
spidermonkey-json	82,294	0.99×	0.80×
shootout-ratelimit	1,092,851	0.99×	0.79×
shootout-minicsv	1,848,205	1.13×	0.79×
shootout-base64	8,429,324	1.00×	0.78×
shootout-matrix	9,028,701	1.01×	0.75×
rust-json	2,925,572	1.18×	0.75×
rust-protobuf	2,894,284	0.98×	0.71×
tinygo-json	68,220	0.99×	0.70×
sqlite3	829,596	1.00×	0.68×
rust-html-rewriter	1,313,456	0.93×	0.67×
spidermonkey-markdown	55,658	0.99×	0.65×
tract-onnx-image-classification	267,646	1.01×	0.63×
shootout-random	6,940,275	1.00×	0.63×
shootout-ed25519	8,424,722	1.00×	0.58×
spidermonkey-regex	27,206	0.97×	0.55×
blind-sig	10,517,630	0.89×	0.40×
shootout-ctype	13,331,031	0.66×	0.37×
noop	0	—	—

doi:10.6342/NTU202601059

Table B.6: Step 2 (Walk-Up + BFS) compile time and `TtV`. Geomean over 39 kernels: `Comp T2/T1` = 2.831×, `Comp LLVM/T2` = 12.021×, `TtV T2/T1` = 1.091×, `TtV LLVM/T2` = 1.648×.

Kernel	Comp T2 (ms)	LLVM/T2 Comp	TtV T2 (ms)	LLVM/T2 TtV
blake3-scalar	94	7.31×	5,330	1.10×
blind-sig	236	13.13×	10,761	0.68×
bz2	63	17.60×	7,254	1.13×
gcc-loops	618	6.03×	8,832	1.20×
hashset	844	2.36×	6,735	1.02×
image-classification	228	11.38×	301	8.86×
pulldown-cmark	178	10.76×	2,765	1.48×
quicksort	21	10.46×	7,017	1.00×
regex	375	13.32×	9,428	1.44×
richards	7	10.17×	916	0.88×
rust-compression	369	15.06×	8,424	1.49×
rust-html-rewriter	392	12.02×	1,718	3.26×
rust-json	164	10.70×	3,096	1.27×
rust-protobuf	121	8.20×	3,020	1.01×
shootout-ackermann	30	10.26×	2,805	1.35×
shootout-base64	25	11.27×	8,455	0.81×
shootout-ctype	24	11.10×	13,355	0.39×
shootout-ed25519	124	20.22×	8,550	0.86×
shootout-fib2	20	10.33×	6,539	0.90×
shootout-gimli	4	4.27×	8,159	0.99×

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	Comp T2 (ms)	LLVM/T2 Comp	TtV T2 (ms)	LLVM/T2 TtV
shootout-heapsort	8	10.83×	9,069	0.89×
shootout-keccak	7	76.46×	6,980	1.06×
shootout-matrix	24	11.44×	9,053	0.78×
shootout-memmove	23	12.58×	2,731	1.07×
shootout-minicsv	6	8.85×	1,855	0.82×
shootout-nestedloop	20	9.95×	5,453	1.67×
shootout-random	21	9.50×	6,961	0.66×
shootout-ratelimit	24	11.59×	1,117	1.02×
shootout-seqhash	23	13.37×	5,442	1.07×
shootout-sieve	20	9.74×	8,923	0.83×
shootout-switch	58	16.33×	8,692	1.15×
shootout-xblabla20	26	10.95×	3,034	0.93×
shootout-xchacha20	22	12.57×	8,458	0.98×
spidermonkey-json	6,308	14.37×	6,577	13.82×
spidermonkey-markdown	6,326	14.22×	6,551	13.76×
spidermonkey-regex	6,314	14.30×	6,504	13.90×
sqlite3	1,064	15.39×	1,913	8.86×
tinygo-json	383	26.90×	463	22.37×
tinygo-regex	295	26.40×	6,396	2.05×
tract-onnx-image-classification	8,279	16.42×	8,990	15.18×
noop	10	—	10	—

doi:10.6342/NTU202601059

B.5 Step 3: Walk-Up Plus BFS Plus OSR

Tier-2 arm from the `osr` branch (commit `bdd80e91`) with the OSR threshold set to 5000. Rows sorted by `LLVM/T2` descending.

Table B.7: Step 3 (Walk-Up + BFS + OSR) WorkTime and ratios. Geomean over 39 kernels: `T1/T2` = 1.105×, `LLVM/T2` = 0.860×.

Kernel	T2 WT (μs)	T1/T2	LLVM/T2
shootout-nestedloop	5,456,438	1.00×	1.63×
image-classification	2,182	1.01×	1.28×
shootout-ackermann	2,752,725	1.71×	1.27×
quicksort	6,544,738	1.23×	1.04×
shootout-gimli	7,929,693	1.02×	1.02×
shootout-seqhash	5,435,950	1.01×	1.02×
shootout-switch	8,998,447	0.94×	1.00×
shootout-xchacha20	7,969,919	1.05×	1.00×
shootout-random	4,383,751	1.58×	1.00×
blake3-scalar	5,256,128	1.02×	0.99×
shootout-keccak	6,935,148	1.20×	0.99×
shootout-base64	6,676,434	1.26×	0.99×
shootout-memmove	2,695,377	1.01×	0.98×
shootout-heapsort	8,213,369	1.11×	0.97×
bz2	7,298,940	1.18×	0.97×
shootout-ratelimit	901,536	1.20×	0.96×

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	T2 WT (μs)	T1/T2	LLVM/T2
shootout-minicsv	1,535,510	1.36×	0.95×
shootout-xblabla20	2,664,736	1.15×	0.95×
shootout-matrix	7,140,199	1.28×	0.95×
regex	9,085,284	1.11×	0.95×
shootout-ctype	5,664,109	1.55×	0.88×
tinygo-regex	6,092,834	1.37×	0.88×
shootout-fib2	6,506,106	1.21×	0.87×
gcc-loops	7,916,342	1.16×	0.86×
rust-compression	8,467,747	1.25×	0.83×
shootout-sieve	8,730,913	0.99×	0.82×
hashset	6,176,044	0.94×	0.80×
blind-sig	5,201,661	1.80×	0.80×
richards	934,762	0.97×	0.78×
pulldown-cmark	2,801,286	1.09×	0.78×
spidermonkey-json	93,950	0.86×	0.70×
rust-protobuf	2,951,811	0.96×	0.70×
tinygo-json	69,082	0.98×	0.69×
shootout-ed25519	7,318,327	1.15×	0.66×
sqlite3	867,947	0.95×	0.65×
rust-json	3,444,532	1.00×	0.63×
tract-onnx-image-classification	271,405	1.00×	0.62×

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	T2 WT (μs)	T1/T2	LLVM/T2
rust-html-rewriter	1,417,953	0.86×	0.62×
spidermonkey-markdown	58,657	0.94×	0.62×
spidermonkey-regex	27,769	0.95×	0.54×
noop	0	—	—

Table B.8: Step 3 (Walk-Up + BFS + OSR) compile time and `TtV`. Geomean over 39 kernels: `Comp T2/T1` = 3.053×, `Comp LLVM/T2` = 11.145×, `TtV T2/T1` = 1.041×, `TtV LLVM/T2` = 1.728×.

Kernel	Comp T2 (ms)	LLVM/T2 Comp	TtV T2 (ms)	LLVM/T2 TtV
blake3-scalar	95	7.21×	5,353	1.10×
blind-sig	259	11.94×	5,462	1.35×
bz2	82	13.39×	7,384	1.11×
gcc-loops	645	5.77×	8,587	1.23×
hashset	419	4.75×	6,623	1.04×
image-classification	235	11.08×	303	8.80×
pulldown-cmark	184	10.39×	2,992	1.37×
quicksort	23	9.55×	6,568	1.07×
regex	401	12.46×	9,507	1.43×
richards	8	8.94×	943	0.86×
rust-compression	415	13.38×	8,896	1.41×
rust-html-rewriter	427	11.04×	1,857	3.01×

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	Comp T2 (ms)	LLVM/T2 Comp	TtV T2 (ms)	LLVM/T2 TtV
rust-json	175	10.04×	3,626	1.09×
rust-protobuf	126	7.86×	3,083	0.99×
shootout-ackermann	33	9.28×	2,786	1.36×
shootout-base64	29	9.84×	6,704	1.03×
shootout-ctype	27	9.69×	5,692	0.93×
shootout-ed25519	136	18.51×	7,456	0.99×
shootout-fib2	22	9.25×	6,529	0.90×
shootout-gimli	4	4.39×	7,934	1.02×
shootout-heapsort	8	9.69×	8,222	0.98×
shootout-keccak	7	76.02×	6,942	1.06×
shootout-matrix	27	10.02×	7,168	0.98×
shootout-memmove	26	11.15×	2,722	1.08×
shootout-minicsv	6	8.89×	1,542	0.99×
shootout-nestedloop	23	8.78×	5,482	1.66×
shootout-random	22	9.30×	4,406	1.04×
shootout-ratelimit	28	9.95×	930	1.23×
shootout-seqhash	28	11.22×	5,464	1.07×
shootout-sieve	21	9.27×	8,753	0.84×
shootout-switch	55	17.33×	9,076	1.10×
shootout-xblabla20	27	10.59×	2,691	1.05×
shootout-xchacha20	26	10.60×	7,997	1.03×

(continued on next page)

doi:10.6342/NTU202601059

(continued from previous page)

Kernel	Comp T2 (ms)	LLVM/T2 Comp	TtV T2 (ms)	LLVM/T2 TtV
spidermonkey-json	8,693	10.43×	8,976	10.13×
spidermonkey-markdown	8,695	10.35×	8,920	10.11×
spidermonkey-regex	8,721	10.35×	8,910	10.15×
sqlite3	1,099	14.90×	1,985	8.54×
tinygo-json	392	26.29×	473	21.93×
tinygo-regex	293	26.58×	6,402	2.05×
tract-onnx-image-classification	8,932	15.22×	9,657	14.13×
noop	12	—	12	—

doi:10.6342/NTU202601059